

CHƯƠNG 15. XỬ LÝ CHUỖI BẰNG RNN VÀ CNN

Dự đoán tương lai là điều bạn làm mọi lúc, cho dù bạn đang kết thúc câu nói của một người bạn hay dự đoán mùi cà phê vào bữa sáng. Trong chương này, chúng ta sẽ thảo luận về mạng thần kinh hồi quy (RNNs) - một loại mạng có thể dự đoán tương lai (tất nhiên, ở một mức độ nhất định). RNNs có thể phân tích dữ liệu chuỗi thời gian, chẳng hạn như số lượng người dùng hoạt động hàng ngày trên trang web của bạn, nhiệt độ hàng giờ trong thành phố của bạn, mức tiêu thụ điện hàng ngày của nhà bạn, quỹ đạo của các xe ô tô gần đó, và nhiều hơn nữa. Khi một RNN học được các mẫu trong dữ liệu quá khứ, nó có thể sử dụng kiến thức của mình để dự báo tương lai, tất nhiên là với giả định rằng các mẫu quá khứ vẫn giữ nguyên trong tương lai.

Tổng quát hơn, RNNs có thể hoạt động trên các chuỗi có độ dài tùy ý, thay vì các đầu vào có kích thước cố định. Ví dụ, chúng có thể nhận câu, tài liệu hoặc mẫu âm thanh làm đầu vào, làm cho chúng cực kỳ hữu ích cho các ứng dụng xử lý ngôn ngữ tự nhiên như dịch tự động hoặc chuyển giọng nói thành văn bản.

Trong chương này, chúng ta sẽ bắt đầu với các khái niệm cơ bản về RNN và cách huấn luyện chúng bằng cách lan truyền ngược qua thời gian (backpropagation through time). Sau đó, chúng ta sẽ sử dụng chúng để dự báo một chuỗi thời gian.

Trong quá trình này, chúng ta sẽ xem xét các mô hình phổ biến thuộc họ ARMA, thường được sử dụng để dự báo chuỗi thời gian và sử dụng chúng làm đường cơ sở để so sánh với RNN của chúng ta. Sau đó, chúng ta sẽ khám phá hai khó khăn chính mà RNN phải đối mặt:

- **Đạo hàm không ổn định** (thảo luận trong Chương 11), có thể được giảm nhẹ bằng các kỹ thuật khác nhau, bao gồm dropout hồi quy và chuẩn hóa lớp hồi quy.
- **Bộ nhớ ngắn hạn (rất) hạn chế**, có thể được mở rộng bằng cách sử dụng các ô LSTM và GRU.

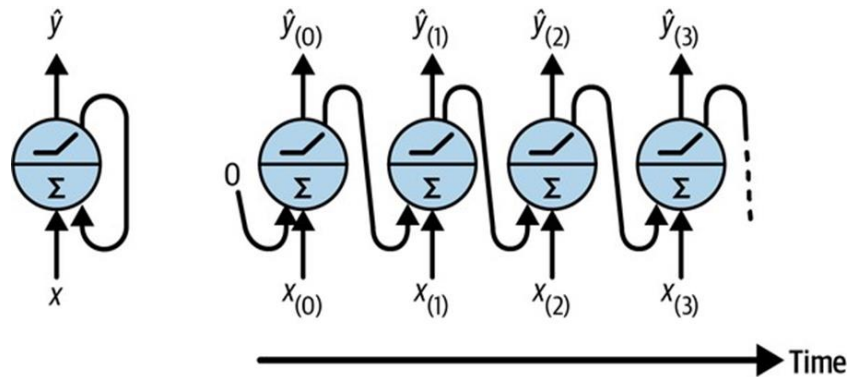
RNN không phải là loại mạng thần kinh duy nhất có khả năng xử lý dữ liệu tuần tự. Đối với các chuỗi nhỏ, một mạng dày đặc thông thường có thể thực hiện được, và đối với các chuỗi rất dài, chẳng hạn như mẫu âm thanh hoặc văn bản, mạng thần kinh tích chập (CNN) cũng có thể hoạt động khá tốt. Chúng ta sẽ thảo luận cả hai khả năng này, và chúng ta sẽ kết thúc chương này bằng cách triển khai WaveNet - một kiến trúc CNN có khả năng xử lý các chuỗi có hàng chục nghìn bước thời gian. Hãy bắt đầu thôi!

15.1 Các nơ-ron và lớp hồi quy

Cho đến nay, chúng ta đã tập trung vào các mạng thần kinh truyền thẳng, nơi các kích hoạt chỉ chảy theo một hướng, từ lớp đầu vào đến lớp đầu ra. Một mạng thần kinh hồi quy trông rất giống một mạng thần kinh truyền thẳng, ngoại trừ việc nó cũng có các kết nối chỉ ngược lại.

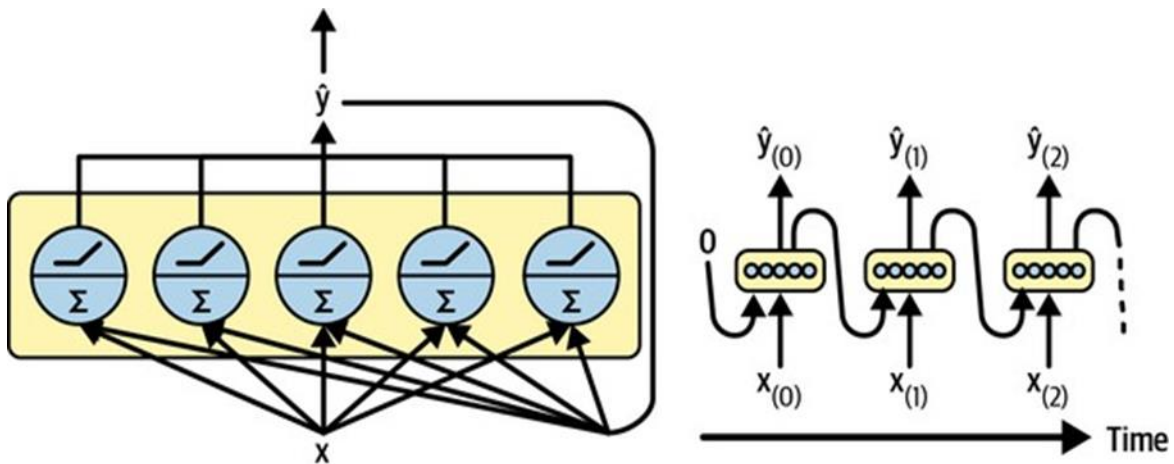
Hãy xem xét RNN đơn giản nhất có thể, bao gồm một nơ-ron nhận đầu vào, tạo ra một đầu ra và gửi đầu ra đó trở lại chính nó, như được thể hiện trong Hình 15-1 (trái). Tại mỗi bước

thời gian t (còn gọi là một khung), nơ-ron hồi quy này nhận các đầu vào $x(t)$ cũng như đầu ra của chính nó từ bước thời gian trước đó, $\hat{y}(t-1)$. Vì không có đầu ra trước đó ở bước thời gian đầu tiên, nó thường được đặt thành 0. Chúng ta có thể biểu diễn mạng nhỏ này trên trục thời gian, như được thể hiện trong Hình 15-1 (phải). Đây được gọi là mở rộng mạng qua thời gian (nó là cùng một nơ-ron hồi quy được biểu diễn một lần cho mỗi bước thời gian).



Hình 15-1. Một nơ-ron hồi quy (trái) được mở rộng qua thời gian (phải)

Bạn có thể dễ dàng tạo một lớp các nơ-ron hồi quy. Tại mỗi bước thời gian t , mỗi nơ-ron nhận cả vector đầu vào $x(t)$ và vector đầu ra từ bước thời gian trước đó $\hat{y}(t-1)$, như được thể hiện trong Hình 15-2. Lưu ý rằng cả đầu vào và đầu ra hiện là các vector (khi chỉ có một nơ-ron duy nhất, đầu ra là một đại lượng vô hướng).



Hình 15-2. Một lớp các nơ-ron hồi quy (trái) được mở rộng qua thời gian (phải)

Mỗi nơ-ron hồi quy có hai tập hợp trọng số: một cho các đầu vào $x(t)$ và một cho các đầu ra của bước thời gian trước đó, $\hat{y}(t-1)$. Hãy gọi các vector trọng số này là w_x và w_y . Nếu chúng ta xem xét toàn bộ lớp hồi quy thay vì chỉ một nơ-ron hồi quy, chúng ta có thể đặt tất cả các vector trọng số vào hai ma trận trọng số: W_x và W_y .

Vector đầu ra của toàn bộ lớp hồi quy sau đó có thể được tính toán như bạn mong đợi, như được thể hiện trong Phương trình 15-1, trong đó b là vector độ lệch và $\phi(\cdot)$ là hàm kích hoạt (ví dụ: ReLU).

Phương trình 15-1. Đầu ra của một lớp hồi quy cho một thể hiện đơn lẻ

$$y^{(t)} = \varphi \left(W_{xT}x^{(t)} + W_{yT}y^{(t-1)} + b \right)$$

Cũng như với các mạng thần kinh truyền thẳng, chúng ta có thể tính toán đầu ra của một lớp hồi quy trong một lần cho toàn bộ một mini-batch bằng cách đặt tất cả các đầu vào tại bước thời gian t vào một ma trận đầu vào $X(t)$ (xem Phương trình 15-2).

Phương trình 15-2. Đầu ra của một lớp nơ-ron hồi quy cho tất cả các thể hiện trong một lượt:

$$\begin{aligned}\hat{Y}_{(t)} &= \phi(\mathbf{X}_{(t)}\mathbf{W}_x + \hat{Y}_{(t-1)}\mathbf{W}_y + \mathbf{b}) \\ &= \phi([\mathbf{X}_{(t)} \quad \hat{Y}_{(t-1)}]\mathbf{W} + \mathbf{b}) \text{ with } \mathbf{W} = \begin{bmatrix} \mathbf{W}_x \\ \mathbf{W}_y \end{bmatrix}\end{aligned}$$

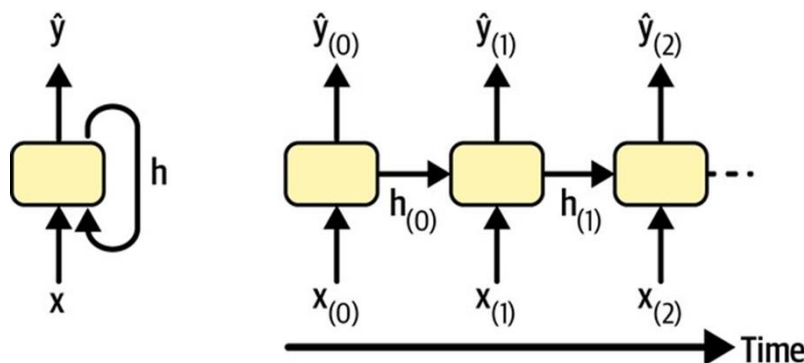
Trong phương trình này:

- $\hat{Y}_{(t)}$ là ma trận $m \times n_{neurons}$ chứa đầu ra của lớp tại thời điểm t cho mỗi trường hợp trong mini-batch (m là số trường hợp trong mini-batch và $n_{neurons}$ là số nơ-ron).
- $X_{(t)}$ là ma trận $m \times n_{inputs}$ chứa các đầu vào cho tất cả các trường hợp (n_{inputs} là số đặc trưng đầu vào).
- W_x là ma trận $n_{inputs} \times n_{neurons}$ chứa các trọng số kết nối cho các đầu vào của thời điểm hiện tại.
- W_y là ma trận $n_{neurons} \times n_{neurons}$ chứa các trọng số kết nối cho các đầu ra của thời điểm trước đó.
- b là một vector có kích thước $n_{neurons}$ chứa số hạng độ lệch của mỗi nơ-ron.
- Các ma trận trọng số W_x và W_y thường được nối theo chiều dọc thành một ma trận trọng số duy nhất W có kích thước $(n_{inputs} + n_{neurons}) \times n_{neurons}$.
- Ký hiệu $[X_{(t)}Y^{(t-1)}]$ đại diện cho việc nối theo chiều ngang của các ma trận $X_{(t)}$ và $Y^{(t-1)}$.
- Lưu ý rằng $Y^{(t)}$ là một hàm của $X_{(t)}$ và $Y^{(t-1)}$, mà $Y^{(t-1)}$ lại là một hàm của $X_{(t-1)}$ và $Y^{(t-2)}$, và cứ tiếp tục như vậy. Điều này làm cho $Y^{(t)}$ trở thành một hàm của tất cả các đầu vào kể từ thời điểm $t = 0$. Tại thời điểm đầu tiên, $t = 0$, không có đầu ra trước đó, vì vậy chúng thường được giả định là tất cả đều bằng không.

Các ô bộ nhớ

Vì đầu ra của một nơ-ron hồi quy tại bước thời gian t là một hàm của tất cả các đầu vào từ các bước thời gian trước đó, bạn có thể nói nó có một dạng bộ nhớ. Một phần của mạng thần kinh giữ lại một số trạng thái qua các bước thời gian được gọi là một ô bộ nhớ (hoặc đơn giản là một ô). Một nơ-ron hồi quy đơn lẻ, hoặc một lớp các nơ-ron hồi quy, là một ô rất cơ bản, chỉ có khả năng học các mẫu ngắn (thường dài khoảng 10 bước, nhưng điều này thay đổi tùy

thuộc vào tác vụ). Sau này trong chương này, chúng ta sẽ xem xét một số loại ô phức tạp và mạnh mẽ hơn có khả năng học các mẫu dài hơn (khoảng 10 lần dài hơn, nhưng một lần nữa, điều này phụ thuộc vào tác vụ). Trạng thái của một ô tại bước thời gian t , được ký hiệu là $h^{(t)}$ (“h” là viết tắt của “ẩn”), là một hàm của một số đầu vào tại bước thời gian đó và trạng thái của nó tại bước thời gian trước đó: $h^{(t)} = f(x^{(t)}, h^{(t-1)})$. Đầu ra của nó tại bước thời gian t , được ký hiệu là $\hat{y}^{(t)}$, cũng là một hàm của trạng thái trước đó và các đầu vào hiện tại. Trong trường hợp các ô cơ bản chúng ta đã thảo luận cho đến nay, đầu ra chỉ bằng trạng thái, nhưng trong các ô phức tạp hơn, điều này không phải lúc nào cũng đúng, như được thể hiện trong Hình 15-3.



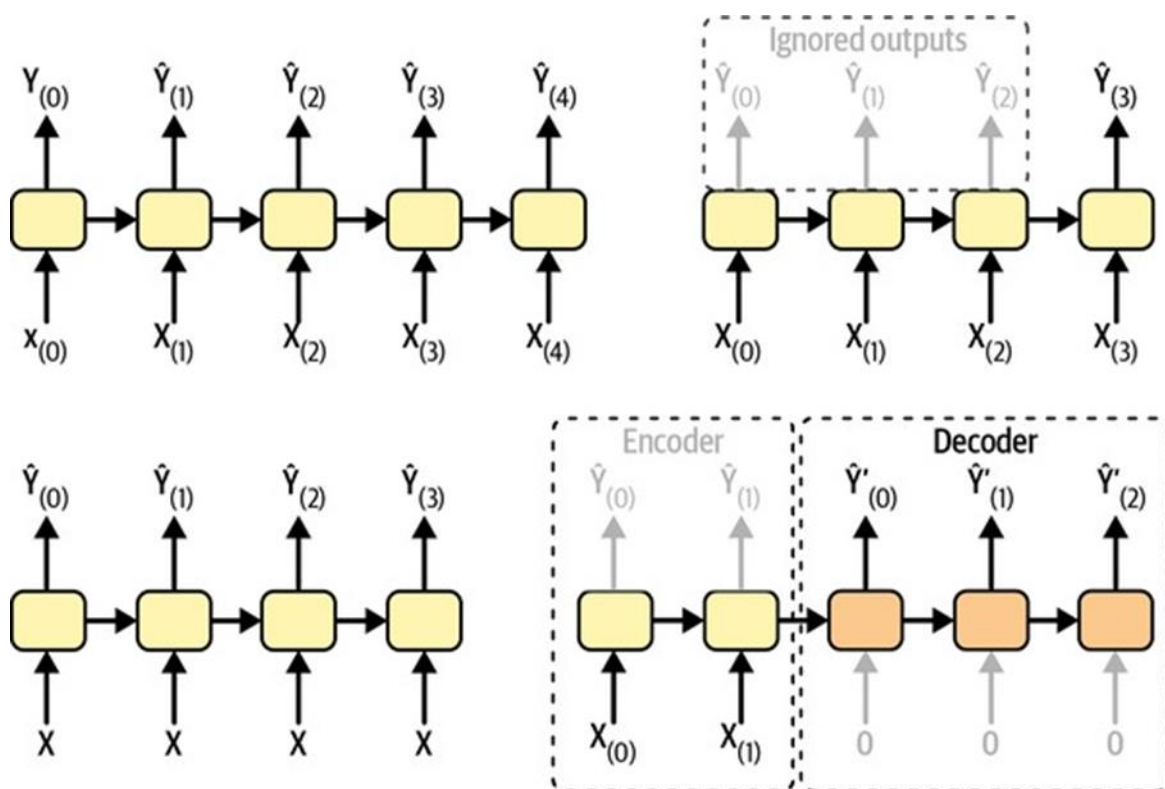
Hình 15-3. Trạng thái ẩn và đầu ra của một ô có thể khác nhau

Các chuỗi đầu vào và đầu ra

Một RNN có thể đồng thời nhận một chuỗi đầu vào và tạo ra một chuỗi đầu ra (xem mạng phía trên bên trái trong Hình 15-4). Loại mạng chuỗi-sang-chuỗi này hữu ích để dự báo chuỗi thời gian, chẳng hạn như mức tiêu thụ điện hàng ngày của nhà bạn: bạn cung cấp dữ liệu trong N ngày qua, và bạn huấn luyện nó để xuất ra mức tiêu thụ điện được dịch chuyển một ngày vào tương lai (tức là, từ $N-1$ ngày trước đến ngày mai). Ngoài ra, bạn có thể cung cấp cho mạng một chuỗi đầu vào và bỏ qua tất cả các đầu ra trừ đầu ra cuối cùng (xem mạng phía trên bên phải trong Hình 15-4). Đây là một mạng chuỗi-sang-vector. Ví dụ, bạn có thể cung cấp cho mạng một chuỗi các từ tương ứng với một bài đánh giá phim, và mạng sẽ xuất ra một điểm tình cảm (ví dụ: từ 0 [ghét] đến 1 [yêu]). Ngược lại, bạn có thể cung cấp cho mạng cùng một vector đầu vào lặp đi lặp lại tại mỗi bước thời gian và để nó xuất ra một chuỗi (xem mạng phía dưới bên trái của Hình 15-4). Đây là một mạng vector-sang-chuỗi. Ví dụ, đầu vào có thể là một hình ảnh (hoặc đầu ra của một CNN), và đầu ra có thể là một chú thích cho hình ảnh đó.

Cuối cùng, bạn có thể có một mạng chuỗi-sang-vector, được gọi là bộ mã hóa (encoder), tiếp theo là một mạng vector-sang-chuỗi, được gọi là bộ giải mã (decoder) (xem mạng phía dưới bên phải của Hình 15-4). Ví dụ, điều này có thể được sử dụng để dịch một câu từ ngôn ngữ này sang ngôn ngữ khác. Bạn sẽ cung cấp cho mạng một câu bằng một ngôn ngữ, bộ mã hóa sẽ chuyển đổi câu này thành một biểu diễn vector duy nhất, và sau đó bộ giải mã sẽ giải mã vector này thành một câu bằng một ngôn ngữ khác. Mô hình hai bước này, được gọi là bộ mã hóa-bộ giải mã, hoạt động tốt hơn nhiều so với việc cố gắng dịch ngay lập tức bằng một

RNN chuỗi-sang-chuỗi đơn lẻ (như cái được biểu diễn ở phía trên bên trái): những từ cuối cùng của một câu có thể ảnh hưởng đến những từ đầu tiên của bản dịch, vì vậy bạn cần đợi cho đến khi bạn đã thấy toàn bộ câu trước khi dịch nó. Chúng ta sẽ tìm hiểu việc triển khai một bộ mã hóa-bộ giải mã trong Chương 16 (như bạn sẽ thấy, nó phức tạp hơn một chút so với những gì Hình 15-4 gợi ý).

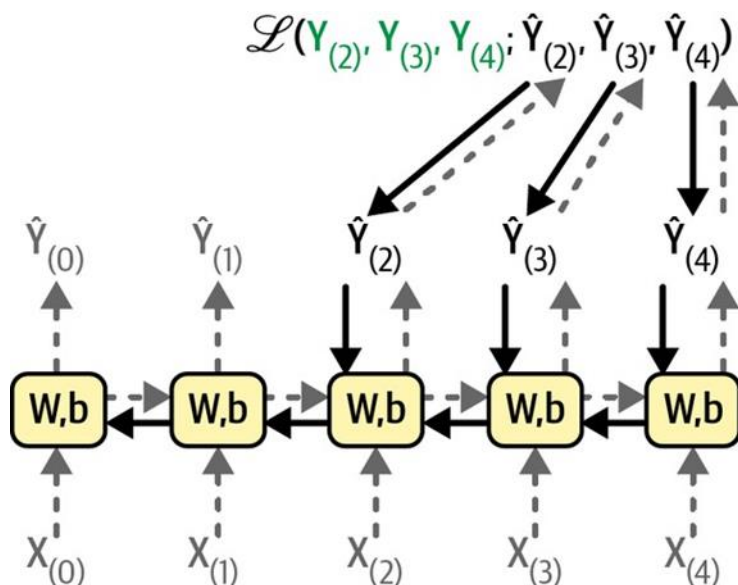


Hình 15-4. Các mạng chuỗi-sang-chuỗi (trên trái), chuỗi-sang-vector (trên phải), vector-sang-chuỗi (dưới trái), và bộ mã hóa-bộ giải mã (dưới phải) Tính linh hoạt này nghe có vẻ hứa hẹn, nhưng làm thế nào để bạn huấn luyện một mạng thần kinh hồi quy?

15.2 Huấn luyện RNN

Để huấn luyện một RNN, cách làm là mở rộng nó theo thời gian (như chúng ta vừa làm) và sau đó sử dụng phương pháp lan truyền ngược thông thường (xem Hình 15-5). Chiến lược này được gọi là **lan truyền ngược qua thời gian (BPTT)**. Giống như trong lan truyền ngược thông thường, có một lượt truyền thẳng đầu tiên qua mạng được mở rộng (được biểu diễn bằng các mũi tên nét đứt). Sau đó, chuỗi đầu ra được đánh giá bằng một hàm mất mát $\text{loss}(Y^{(0)}, Y^{(1)}, \dots, Y^{(T)}; \hat{y}^{(0)}, \hat{y}^{(1)}, \dots, \hat{y}^{(T)})$ (trong đó $Y^{(i)}$ là mục tiêu thứ i , $\hat{y}^{(i)}$ là dự đoán thứ i , và T là bước thời gian tối đa). Lưu ý rằng hàm mất mát này có thể bỏ qua một số đầu ra. Ví dụ, trong một RNN chuỗi-sang-vector, tất cả các đầu ra đều bị bỏ qua trừ đầu ra cuối cùng. Trong Hình 15-5, hàm mất mát được tính toán chỉ dựa trên ba đầu ra cuối cùng. Các đạo hàm của hàm mất mát đó sau đó được lan truyền ngược qua mạng được mở rộng (được biểu diễn bằng các mũi tên liền nét). Trong ví dụ này, vì các đầu ra $\hat{y}^{(0)}$ và $\hat{y}^{(1)}$ không được sử dụng để tính toán mất mát, các đạo hàm không chảy ngược qua chúng; chúng chỉ chảy qua

$\hat{Y}^{(2)}$, $\hat{Y}^{(3)}$ và $\hat{Y}^{(4)}$. Hơn nữa, vì cùng các tham số W và b được sử dụng ở mỗi bước thời gian, các đạo hàm của chúng sẽ được điều chỉnh nhiều lần trong quá trình lan truyền ngược. Sau khi giai đoạn lan truyền ngược hoàn tất và tất cả các đạo hàm đã được tính toán, BPTT có thể thực hiện một bước giảm độ dốc để cập nhật các tham số (điều này không khác gì lan truyền ngược thông thường).



Hình 15-5. Lan truyền ngược qua thời gian

May mắn thay, Keras sẽ lo tất cả sự phức tạp này cho bạn, như bạn sẽ thấy. Nhưng trước khi chúng ta đến đó, hãy tải một chuỗi thời gian và bắt đầu phân tích nó bằng các công cụ cổ điển để hiểu rõ hơn những gì chúng ta đang xử lý và để có được một số số liệu cơ bản.

15.3 Dự báo chuỗi thời gian

Được rồi! Giả sử bạn vừa được thuê làm nhà khoa học dữ liệu bởi Cơ quan Vận tải Chicago. Nhiệm vụ đầu tiên của bạn là xây dựng một mô hình có khả năng dự báo số lượng hành khách sẽ đi xe buýt và đường sắt vào ngày hôm sau. Bạn có quyền truy cập vào dữ liệu lượng hành khách hàng ngày kể từ năm 2001. Hãy cùng nhau tìm hiểu cách bạn sẽ xử lý việc này. Chúng ta sẽ bắt đầu bằng cách tải và làm sạch dữ liệu:

```
path = Path("datasets/ridership/CTA_-_Ridership_-_Daily_Boarding_Totals.csv")
df = pd.read_csv(path, parse_dates=["service_date"])
df.columns = ["date", "day_type", "bus", "rail", "total"] # shorter names
df = df.sort_values("date").set_index("date")
df = df.drop("total", axis=1) # no need for total, it's just bus + rail
df = df.drop_duplicates() # remove duplicated months (2011-10 and 2014-07)
```

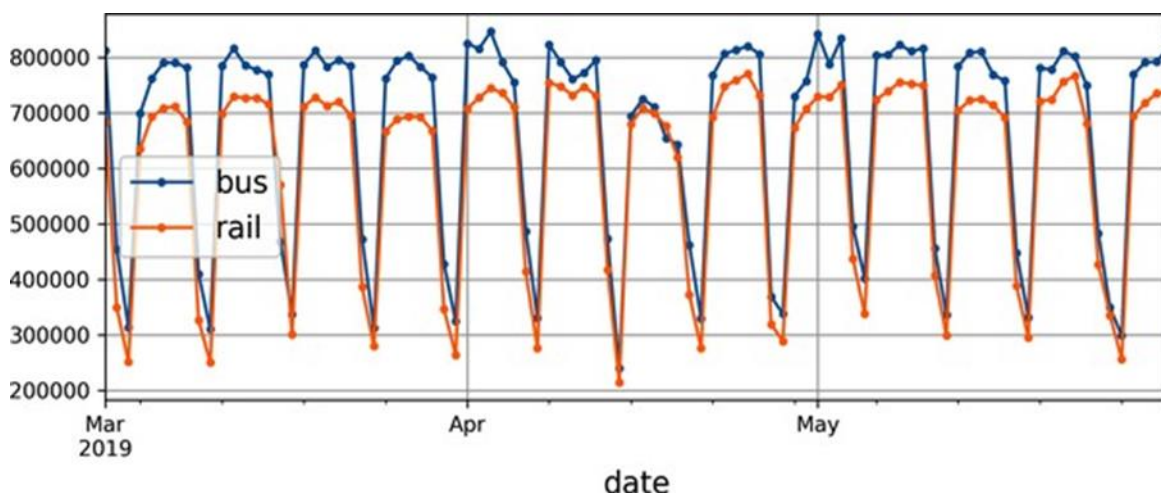
Chúng ta tải tệp CSV, đặt tên cột ngắn gọn, sắp xếp các hàng theo ngày, loại bỏ cột “total” thừa và loại bỏ các hàng trùng lặp. Bây giờ hãy kiểm tra vài hàng đầu tiên trông như thế nào:

```
>>> df.head()
          day_type  bus  rail
```


date				
2001-01-01	U	297192	126455	
2001-01-02	W	780827	501952	
2001-01-03	W	824923	536432	
2001-01-04	W	870021	550011	
2001-01-05	W	890426	557917	

Vào ngày 1 tháng 1 năm 2001, 297.192 người đã đi xe buýt ở Chicago và 126.455 người đã đi tàu. Cột `day_type` chứa W cho Ngày trong tuần, A cho Thứ Bảy và U cho Chủ Nhật hoặc ngày lễ. Bây giờ chúng ta hãy vẽ biểu đồ số lượng hành khách đi xe buýt và đường sắt trong vài tháng vào năm 2019, để xem nó trông như thế nào (xem Hình 15-6):

```
df["2019-03":"2019-05"].plot(grid=True, marker=".", figsize=(8, 3.5))
plt.show()
```



Hình 15-6. Lượng hành khách hàng ngày ở Chicago

Lưu ý rằng Pandas bao gồm cả tháng bắt đầu và tháng kết thúc trong phạm vi, vì vậy biểu đồ này vẽ dữ liệu từ ngày 1 tháng 3 cho đến ngày 31 tháng 5. Đây là một **chuỗi thời gian**: dữ liệu có các giá trị tại các bước thời gian khác nhau, thường là ở các khoảng thời gian đều đặn. Cụ thể hơn, vì có nhiều giá trị trên mỗi bước thời gian, đây được gọi là **chuỗi thời gian đa biến**. Nếu chúng ta chỉ nhìn vào cột “bus”, nó sẽ là một chuỗi thời gian đơn biến, với một giá trị duy nhất trên mỗi bước thời gian. Dự đoán các giá trị tương lai (tức là **dự báo**) là nhiệm vụ điển hình nhất khi xử lý chuỗi thời gian, và đây là điều chúng ta sẽ tập trung vào trong chương này. Các nhiệm vụ khác bao gồm **điền dữ liệu** (điền các giá trị quá khứ bị thiếu), **phân loại**, **phát hiện bất thường**, v.v. Nhìn vào Hình 15-6, chúng ta có thể thấy rằng một mô hình tương tự được lặp lại rõ ràng mỗi tuần. Đây được gọi là **tính thời vụ hàng tuần**. Trên thực tế, trong trường hợp này nó rất mạnh đến nỗi việc dự báo lượng hành khách ngày mai bằng cách chỉ sao chép các giá trị từ một tuần trước sẽ cho kết quả khá tốt. Đây được gọi là **dự báo ngày thơ**: chỉ đơn giản là sao chép một giá trị trong quá khứ để thực hiện dự báo của chúng ta. Dự báo ngày thơ thường là một đường cơ sở tuyệt vời, và đôi khi thậm chí khó bị đánh bại.

Để hình dung các dự báo ngày thơ này, chúng ta hãy phủ lên hai chuỗi thời gian (đối với xe buýt và đường sắt) cũng như cùng một chuỗi thời gian bị trễ một tuần (tức là dịch chuyển

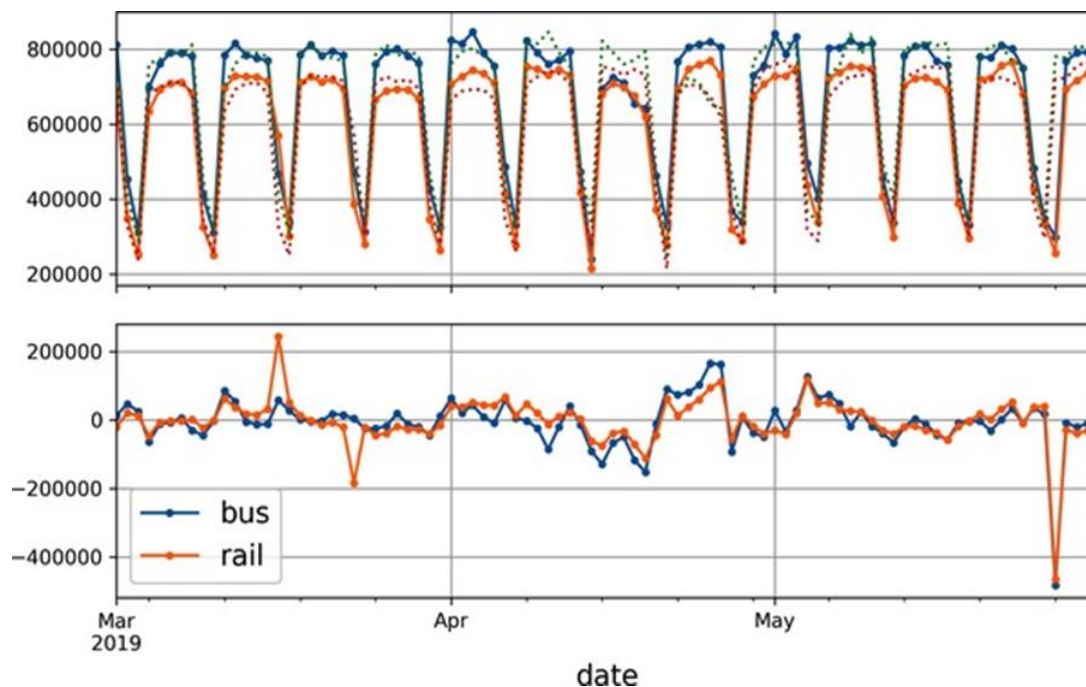
sang phải) bằng các đường chấm chấm. Chúng ta cũng sẽ vẽ biểu đồ sự khác biệt giữa hai chuỗi (tức là giá trị tại thời điểm t trừ đi giá trị tại thời điểm $t - 7$); đây được gọi là **sai phân** (xem Hình 15-7):

```
diff_7 = df[["bus", "rail"]].diff(7)["2019-03":"2019-05"]

fig, axs = plt.subplots(2, 1, sharex=True, figsize=(8, 5))
df.plot(ax=axs[0], legend=False, marker=".") # original time series
df.shift(7).plot(ax=axs[0], grid=True, legend=False, linestyle=":") # lagged
diff_7.plot(ax=axs[1], grid=True, marker=".") # 7-day difference time series
plt.show()
```

Không tệ lắm! Lưu ý rằng chuỗi thời gian bị trễ theo dõi chặt chẽ chuỗi thời gian thực tế như thế nào. Khi một chuỗi thời gian tương quan với một phiên bản trễ của chính nó, chúng ta nói rằng chuỗi thời gian đó có **tự tương quan**. Như bạn có thể thấy, hầu hết các khác biệt đều khá nhỏ, ngoại trừ vào cuối tháng Năm. Có lẽ có một kỳ nghỉ vào thời điểm đó? Hãy kiểm tra cột `day_type`:

```
>>> list(df.loc["2019-05-25":"2019-05-27"]["day_type"])
['A', 'U', 'U']
```



Hình 15-7. Chuỗi thời gian được phủ với chuỗi thời gian bị trễ 7 ngày (trên), và sự khác biệt giữa t và $t - 7$ (dưới)

Thật vậy, lúc đó có một kỳ nghỉ cuối tuần dài: Thứ Hai là ngày lễ Tưởng niệm. Chúng ta có thể sử dụng cột này để cải thiện dự báo của mình, nhưng bây giờ chúng ta hãy chỉ đo lỗi tuyệt đối trung bình trong khoảng thời gian ba tháng mà chúng ta đang tập trung vào—tháng 3, tháng 4 và tháng 5 năm 2019—để có một ý tưởng sơ bộ:


```
>>> diff_7.abs().mean()
bus      43915.608696
rail     42143.271739
dtype: float64
```

Các dự báo ngày thơ của chúng ta có MAE khoảng 43.916 hành khách xe buýt và khoảng 42.143 hành khách đường sắt. Rất khó để nói ngay lập tức điều này tốt hay xấu, vì vậy hãy đặt các lỗi dự báo vào góc nhìn bằng cách chia chúng cho các giá trị mục tiêu:

```
targets = df[["bus", "rail"]]["2019-03":"2019-05"]
(diff_7 / targets).abs().mean()

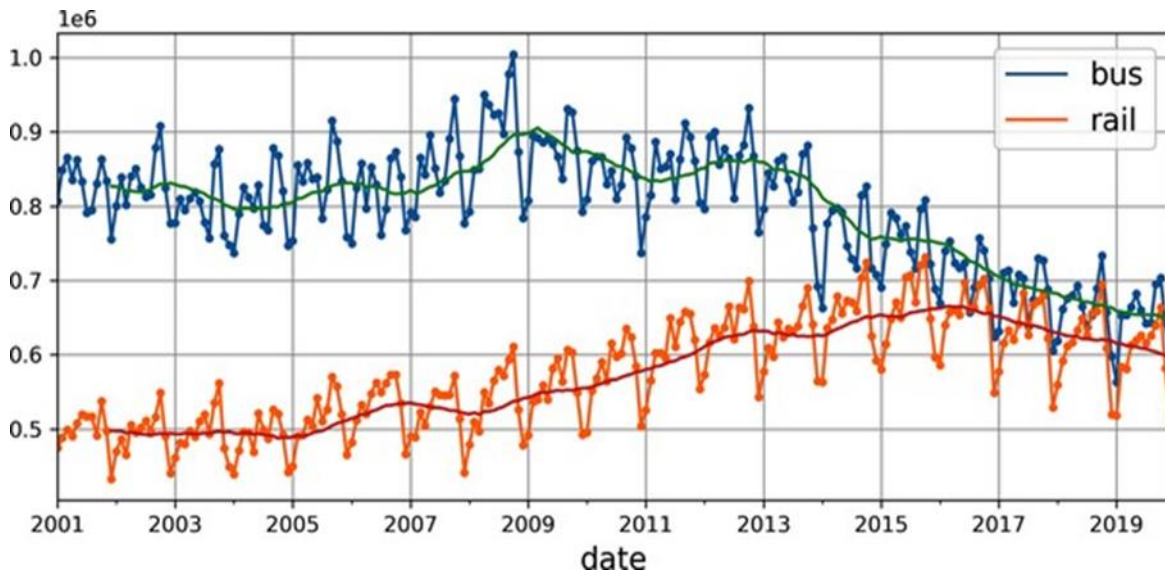
bus      0.082938
rail     0.089948
dtype: float64
```

Những gì chúng ta vừa tính toán được gọi là **lỗi phần trăm tuyệt đối trung bình (MAPE)**: có vẻ như các dự báo ngày thơ của chúng ta cho chúng ta MAPE khoảng 8,3% đối với xe buýt và 9,0% đối với đường sắt. Điều thú vị là MAE cho các dự báo đường sắt trông hơi tốt hơn MAE cho các dự báo xe buýt, trong khi điều ngược lại đúng với MAPE. Điều đó là do lượng hành khách xe buýt lớn hơn lượng hành khách đường sắt, vì vậy tự nhiên lỗi dự báo cũng lớn hơn, nhưng khi chúng ta đặt các lỗi vào góc nhìn, hóa ra các dự báo xe buýt thực sự hơi tốt hơn các dự báo đường sắt.

Nhìn vào chuỗi thời gian, dường như không có bất kỳ tính thời vụ hàng tháng đáng kể nào, nhưng hãy kiểm tra xem có bất kỳ tính thời vụ hàng năm nào không. Chúng ta sẽ xem xét dữ liệu từ năm 2001 đến năm 2019. Để giảm nguy cơ dò dữ liệu, chúng ta sẽ bỏ qua dữ liệu gần đây hơn. Chúng ta cũng sẽ vẽ một đường trung bình động 12 tháng cho mỗi chuỗi để hình dung các xu hướng dài hạn (xem Hình 15-8):

```
period = slice("2001", "2019")
df_monthly = df.resample('M').mean() # compute the mean for each month
rolling_average_12_months = df_monthly[period].rolling(window=12).mean()

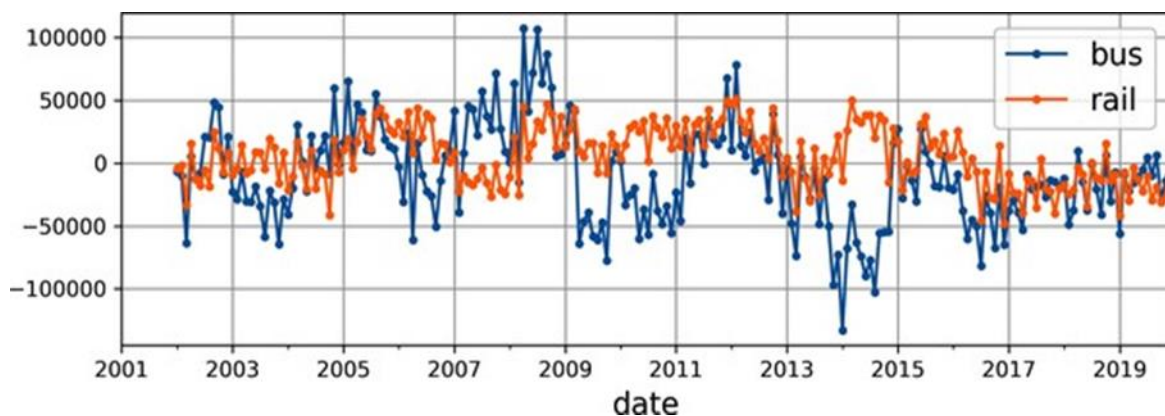
fig, ax = plt.subplots(figsize=(8, 4))
df_monthly[period].plot(ax=ax, marker=".")
rolling_average_12_months.plot(ax=ax, grid=True, legend=False)
plt.show()
```



Hình 15-8. Tính thời vụ hàng năm và các xu hướng dài hạn

Đúng vậy! Chắc chắn có một số **tính thời vụ hàng năm** nữa, mặc dù nó nhiều hơn tính thời vụ hàng tuần và rõ ràng hơn đối với chuỗi đường sắt so với chuỗi xe buýt: chúng ta thấy các đỉnh và đáy vào cùng những ngày gần nhau mỗi năm. Hãy kiểm tra xem chúng ta nhận được gì nếu chúng ta vẽ biểu đồ sự khác biệt 12 tháng (xem Hình 15-9):

```
df_monthly.diff(12)[period].plot(grid=True, marker=".", figsize=(8, 3))
plt.show()
```



Hình 15-9. Sự khác biệt 12 tháng

Lưu ý rằng việc **sai phân** không chỉ loại bỏ tính thời vụ hàng năm, mà còn loại bỏ các xu hướng dài hạn. Ví dụ, xu hướng giảm tuyến tính hiện diện trong chuỗi thời gian từ năm 2016 đến 2019 đã trở thành một giá trị âm gần như không đổi trong chuỗi thời gian đã được sai phân. Trên thực tế, sai phân là một kỹ thuật phổ biến được sử dụng để loại bỏ xu hướng và tính thời vụ khỏi một chuỗi thời gian: việc nghiên cứu một **chuỗi thời gian tĩnh** dễ dàng hơn, nghĩa là một chuỗi có các thuộc tính thống kê không đổi theo thời gian, không có bất kỳ tính thời vụ hoặc xu hướng nào. Khi bạn có thể đưa ra các dự báo chính xác trên chuỗi thời gian đã được sai phân, việc biến chúng thành các dự báo cho chuỗi thời gian thực tế rất dễ dàng bằng

cách chỉ cần cộng lại các giá trị trong quá khứ đã bị trừ trước đó. Bạn có thể nghĩ rằng chúng ta chỉ đang cố gắng dự đoán lượng hành khách vào ngày mai, vì vậy các mẫu dài hạn ít quan trọng hơn nhiều so với các mẫu ngắn hạn. Bạn đúng, nhưng dù sao đi nữa, chúng ta vẫn có thể cải thiện hiệu suất một chút bằng cách tính đến các mẫu dài hạn. Ví dụ, lượng hành khách xe buýt hàng ngày đã giảm khoảng 2.500 vào tháng 10 năm 2017, đại diện cho khoảng 570 hành khách ít hơn mỗi tuần, vì vậy nếu chúng ta đang ở cuối tháng 10 năm 2017, việc dự báo lượng hành khách vào ngày mai bằng cách sao chép giá trị từ tuần trước, trừ đi 570 sẽ có ý nghĩa. Việc tính đến xu hướng sẽ làm cho dự báo của bạn chính xác hơn một chút so với mức trung bình. Bây giờ bạn đã quen thuộc với chuỗi thời gian lượng hành khách, cũng như một số khái niệm quan trọng nhất trong phân tích chuỗi thời gian, bao gồm tính thời vụ, xu hướng, sai phân và trung bình động, hãy cùng xem nhanh một họ các mô hình thống kê rất phổ biến thường được sử dụng để phân tích chuỗi thời gian.

Các mô hình họ ARMA

Chúng ta sẽ bắt đầu với mô hình trung bình động tự hồi quy (ARMA), được Herman Wold phát triển vào những năm 1930: nó tính toán các dự báo của mình bằng cách sử dụng tổng trọng số đơn giản của các giá trị bị trễ và hiệu chỉnh các dự báo này bằng cách thêm một trung bình động, rất giống như chúng ta vừa thảo luận. Cụ thể, thành phần trung bình động được tính toán bằng cách sử dụng tổng trọng số của một vài lỗi dự báo gần nhất. Phương trình 15-3 cho thấy cách mô hình đưa ra các dự báo của nó.

Phương trình 15-3. Dự báo bằng mô hình ARMA

$$y^{(t)} = \sum_{i=1}^p \alpha_i y^{(t-i)} + \sum_{i=1}^q \theta_i \epsilon^{(t-i)}$$

với $\epsilon^{(t)} = y^{(t)} - \hat{y}^{(t)}$ Trong phương trình này:

- $\hat{y}^{(t)}$ là dự báo của mô hình cho bước thời gian t .
- $y^{(t)}$ là giá trị của chuỗi thời gian tại bước thời gian t .
- Tổng đầu tiên là tổng trọng số của p giá trị quá khứ của chuỗi thời gian, sử dụng các trọng số đã học α_i . Số p là một siêu tham số, và nó xác định mô hình nên nhìn lại quá khứ bao xa. Tổng này là thành phần tự hồi quy của mô hình: nó thực hiện hồi quy dựa trên các giá trị trong quá khứ.
- Tổng thứ hai là tổng trọng số trên q lỗi dự báo quá khứ $\epsilon^{(t)}$, sử dụng các trọng số đã học θ_i . Số q là một siêu tham số. Tổng này là thành phần trung bình động của mô hình.

Quan trọng là, mô hình này giả định rằng chuỗi thời gian là **tĩnh (stationary)**. Nếu không, thì việc sai phân có thể giúp ích. Sử dụng sai phân trên một bước thời gian duy nhất sẽ tạo ra một xấp xỉ của đạo hàm chuỗi thời gian: thực sự, nó sẽ cho độ dốc của chuỗi tại mỗi bước thời gian. Điều này có nghĩa là nó sẽ loại bỏ bất kỳ xu hướng tuyến tính nào, biến nó thành một giá trị không đổi. Ví dụ, nếu bạn áp dụng sai phân một bước cho chuỗi [3, 5, 7, 9, 11], bạn sẽ nhận được chuỗi sai phân [2, 2, 2, 2]. Nếu chuỗi thời gian gốc có xu hướng bậc hai

thay vì xu hướng tuyến tính, thì một lần sai phân sẽ không đủ. Ví dụ, chuỗi [1, 4, 9, 16, 25, 36] trở thành [3, 5, 7, 9, 11] sau một lần sai phân, nhưng nếu bạn chạy sai phân lần thứ hai, thì bạn sẽ nhận được [2, 2, 2, 2]. Vì vậy, chạy hai lần sai phân liên tiếp sẽ loại bỏ các xu hướng bậc hai. Tổng quát hơn, chạy d lần sai phân liên tiếp tính toán một xấp xỉ của đạo hàm bậc d của chuỗi thời gian, vì vậy nó sẽ loại bỏ các xu hướng đa thức bậc lên đến d . Siêu tham số d này được gọi là **bậc tích hợp**.

Sai phân là đóng góp trung tâm của mô hình **trung bình động tích hợp tự hồi quy (ARIMA)**, được giới thiệu vào năm 1970 bởi George Box và Gwilym Jenkins trong cuốn sách *Time Series Analysis* của họ (Wiley): mô hình này chạy d lần sai phân để làm cho chuỗi thời gian trở nên tĩnh hơn, sau đó nó áp dụng một mô hình ARMA thông thường. Khi đưa ra dự báo, nó sử dụng mô hình ARMA này, sau đó nó cộng lại các thuật ngữ đã bị trừ đi bởi sai phân. Một thành viên cuối cùng của họ ARMA là mô hình **ARIMA mùa vụ (SARIMA)**: nó mô hình chuỗi thời gian theo cùng một cách như ARIMA, nhưng nó bổ sung thêm mô hình một thành phần mùa vụ cho một tần số nhất định (ví dụ: hàng tuần), sử dụng cùng một phương pháp ARIMA chính xác. Nó có tổng cộng bảy siêu tham số: các siêu tham số p, d , và q giống như ARIMA, cộng với các siêu tham số P, D , và Q bổ sung để mô hình mẫu mùa vụ, và cuối cùng là chu kỳ của mẫu mùa vụ, được ký hiệu là s . Các siêu tham số P, D , và Q giống như p, d , và q , nhưng chúng được sử dụng để mô hình chuỗi thời gian tại $t - s, t - 2s, t - 3s$, v.v.

Hãy xem cách điều chỉnh mô hình SARIMA cho chuỗi thời gian đường sắt và sử dụng nó để đưa ra dự báo về lượng hành khách vào ngày mai. Chúng ta sẽ giả vờ hôm nay là ngày cuối cùng của tháng 5 năm 2019, và chúng ta muốn dự báo lượng hành khách đường sắt cho “ngày mai”, ngày 1 tháng 6 năm 2019. Để làm điều này, chúng ta có thể sử dụng thư viện statsmodels, chứa nhiều mô hình thống kê khác nhau, bao gồm mô hình ARMA và các biến thể của nó, được triển khai bởi lớp ARIMA:

```
from statsmodels.tsa.arima.model import ARIMA

origin, today = "2019-01-01", "2019-05-31"
rail_series = df.loc[origin:today]["rail"].asfreq("D")
model = ARIMA(rail_series,
               order=(1, 0, 0),
               seasonal_order=(0, 1, 1, 7))
model = model.fit()
y_pred = model.forecast() # returns 427,758.6
```

Trong ví dụ mã này:

- Chúng ta bắt đầu bằng cách nhập lớp ARIMA, sau đó chúng ta lấy dữ liệu lượng hành khách đường sắt từ đầu năm 2019 cho đến “ngày hôm nay”, và chúng ta sử dụng `asfreq("D")` để đặt tần suất của chuỗi thời gian thành hàng ngày: điều này không thay đổi dữ liệu trong trường hợp này, vì nó đã là hàng ngày, nhưng nếu không có điều này, lớp ARIMA sẽ phải đoán tần suất và nó sẽ hiển thị cảnh báo.
- Tiếp theo, chúng ta tạo một thể hiện ARIMA, truyền cho nó tất cả dữ liệu cho đến “ngày hôm nay”, và chúng ta đặt các siêu tham số mô hình: `order=(1, 0, 0)` có nghĩa là

$p = 1, d = 0, q = 0$, và $\text{seasonal_order}=(0, 1, 1, 7)$ có nghĩa là $P = 0, D = 1, Q = 1$, và $s = 7$. Lưu ý rằng API của `statsmodels` hơi khác so với API của Scikit-Learn, vì chúng ta truyền dữ liệu cho mô hình tại thời điểm khởi tạo, thay vì truyền nó cho phương thức `fit()`.

- Tiếp theo, chúng ta điều chỉnh mô hình và sử dụng nó để đưa ra dự báo cho “ngày mai”, ngày 1 tháng 6 năm 2019.

Dự báo là 427.759 hành khách, trong khi thực tế có 379.044. Chà, chúng ta đã sai lệch 12,9%—khá tệ. Nó thực sự tệ hơn một chút so với dự báo ngây thơ, dự báo 426.932, sai lệch 12,6%. Nhưng có lẽ chúng ta chỉ không may mắn vào ngày hôm đó? Để kiểm tra điều này, chúng ta có thể chạy cùng một mã trong một vòng lặp để đưa ra dự báo cho mỗi ngày trong tháng 3, tháng 4 và tháng 5, và tính MAE trong khoảng thời gian đó:

```
origin, start_date, end_date = "2019-01-01", "2019-03-01", "2019-05-31"
time_period = pd.date_range(start_date, end_date)
rail_series = df.loc[origin:end_date]["rail"].asfreq("D")
y_preds = []

for today in time_period.shift(-1):
    model = ARIMA(rail_series[origin:today], # train on data up to "today"
                  order=(1, 0, 0),
                  seasonal_order=(0, 1, 1, 7))
    model = model.fit() # note that we retrain the model every day!
    y_pred = model.forecast()[0]
    y_preds.append(y_pred)

y_preds = pd.Series(y_preds, index=time_period)
mae = (y_preds - rail_series[time_period]).abs().mean() # returns 32,040.7
```

À, tốt hơn nhiều! MAE khoảng 32.041, thấp hơn đáng kể so với MAE chúng ta nhận được với dự báo ngây thơ (42.143). Vì vậy, mặc dù mô hình không hoàn hảo, nhưng nó vẫn đánh bại dự báo ngây thơ với một biên độ lớn, trung bình. Tại thời điểm này, bạn có thể tự hỏi làm thế nào để chọn các siêu tham số tốt cho mô hình SARIMA. Có một số phương pháp, nhưng phương pháp đơn giản nhất để hiểu và bắt đầu là phương pháp vét cạn: chỉ cần chạy tìm kiếm lưới (grid search). Đối với mỗi mô hình bạn muốn đánh giá (tức là mỗi sự kết hợp siêu tham số), bạn có thể chạy ví dụ mã trước đó, chỉ thay đổi các giá trị siêu tham số. Các giá trị p, q, P , và Q tốt thường khá nhỏ (thường từ 0 đến 2, đôi khi lên đến 5 hoặc 6), và d và D thường là 0 hoặc 1, đôi khi là 2. Đối với s , nó chỉ là chu kỳ của mẫu mùa vụ chính: trong trường hợp của chúng ta là 7 vì có tính thời vụ hàng tuần mạnh mẽ. Mô hình có MAE thấp nhất sẽ thắng. Tất nhiên, bạn có thể thay thế MAE bằng một chỉ số khác nếu nó phù hợp hơn với mục tiêu kinh doanh của bạn. Và thế là xong!

Chuẩn bị dữ liệu cho các mô hình học máy

Bây giờ chúng ta có hai đường cơ sở, dự báo ngây thơ và SARIMA, hãy thử sử dụng các mô hình học máy mà chúng ta đã tìm hiểu cho đến nay để dự báo chuỗi thời gian này, bắt đầu với một mô hình tuyến tính cơ bản. Mục tiêu của chúng ta sẽ là dự báo lượng hành khách vào

ngày mai dựa trên dữ liệu lượng hành khách của 8 tuần qua (56 ngày). Do đó, đầu vào của mô hình của chúng ta sẽ là các chuỗi (thường là một chuỗi duy nhất mỗi ngày khi mô hình đi vào sản xuất), mỗi chuỗi chứa 56 giá trị từ bước thời gian $t - 55$ đến t . Đối với mỗi chuỗi đầu vào, mô hình sẽ xuất ra một giá trị duy nhất: dự báo cho bước thời gian $t + 1$. Nhưng chúng ta sẽ sử dụng gì làm dữ liệu huấn luyện? À, đó là mẹo: chúng ta sẽ sử dụng mọi cửa sổ 56 ngày từ quá khứ làm dữ liệu huấn luyện, và mục tiêu cho mỗi cửa sổ sẽ là giá trị ngay sau đó. Keras thực sự có một hàm tiện ích hay tên là `tf.keras.utils.timeseries_dataset_from_array()` để giúp chúng ta chuẩn bị tập huấn luyện. Nó nhận một chuỗi thời gian làm đầu vào, và nó xây dựng một `tf.data.Dataset` (được giới thiệu trong Chương 13) chứa tất cả các cửa sổ có độ dài mong muốn, cũng như các mục tiêu tương ứng của chúng. Đây là một ví dụ lấy một chuỗi thời gian chứa các số từ 0 đến 5 và tạo một tập dữ liệu chứa tất cả các cửa sổ có độ dài 3, với các mục tiêu tương ứng của chúng, được nhóm thành các batch có kích thước 2:

```
import tensorflow as tf

my_series = [0, 1, 2, 3, 4, 5]
my_dataset = tf.keras.utils.timeseries_dataset_from_array(
    my_series,
    targets=my_series[3:], # the targets are 3 steps into the future
    sequence_length=3,
    batch_size=2
)
```

Hãy kiểm tra nội dung của tập dữ liệu này:

```
>>> list(my_dataset)
[(<tf.Tensor: shape=(2, 3), dtype=int32, numpy=
  array([[0, 1, 2],
        [1, 2, 3]], dtype=int32)>,
  <tf.Tensor: shape=(2,), dtype=int32, numpy=array([3, 4], dtype=int32)>),
 (<tf.Tensor: shape=(1, 3), dtype=int32, numpy=array([[2, 3, 4]], dtype=int32)
 >),
  <tf.Tensor: shape=(1,), dtype=int32, numpy=array([5], dtype=int32)>)]
```

Mỗi mẫu trong tập dữ liệu là một cửa sổ có độ dài 3, cùng với mục tiêu tương ứng của nó (tức là giá trị ngay sau cửa sổ). Các cửa sổ là [0, 1, 2], [1, 2, 3] và [2, 3, 4], và các mục tiêu tương ứng của chúng là 3, 4 và 5. Vì có tổng cộng ba cửa sổ, không phải là bội số của kích thước batch, nên batch cuối cùng chỉ chứa một cửa sổ thay vì hai. Một cách khác để có được kết quả tương tự là sử dụng phương thức `window()` của lớp `tf.data.Dataset`. Nó phức tạp hơn, nhưng nó cho bạn toàn quyền kiểm soát, điều này sẽ hữu ích sau này trong chương này, vì vậy hãy xem nó hoạt động như thế nào. Phương thức `window()` trả về một tập dữ liệu của các tập dữ liệu cửa sổ:

```
>>> for window_dataset in tf.data.Dataset.range(6).window(4, shift=1):
...     for element in window_dataset:
...         print(f"{element}", end=" ")
...     print()
```

```
...
0 1 2 3
1 2 3 4
2 3 4 5
3 4 5
4 5
5
```

Trong ví dụ này, tập dữ liệu chứa sáu cửa sổ, mỗi cửa sổ được dịch chuyển một bước so với cửa sổ trước đó, và ba cửa sổ cuối cùng nhỏ hơn vì chúng đã đạt đến cuối chuỗi. Nói chung, bạn sẽ muốn loại bỏ các cửa sổ nhỏ hơn này bằng cách truyền `drop_remainder=True` cho phương thức `window()`.

Phương thức `window()` trả về một tập dữ liệu lồng nhau, tương tự như một danh sách các danh sách. Điều này hữu ích khi bạn muốn biến đổi mỗi cửa sổ bằng cách gọi các phương thức tập dữ liệu của nó (ví dụ: để xáo trộn hoặc nhóm chúng). Tuy nhiên, chúng ta không thể sử dụng trực tiếp tập dữ liệu lồng nhau để huấn luyện, vì mô hình của chúng ta sẽ mong đợi các tensor làm đầu vào, không phải tập dữ liệu. Do đó, chúng ta phải gọi phương thức `flat_map()`: nó chuyển đổi một tập dữ liệu lồng nhau thành một tập dữ liệu phẳng (một tập dữ liệu chứa các tensor, không phải tập dữ liệu). Ví dụ, giả sử `{1, 2, 3}` đại diện cho một tập dữ liệu chứa chuỗi các tensor 1, 2 và 3. Nếu bạn làm phẳng tập dữ liệu lồng nhau `{{1, 2}, {3, 4, 5, 6}}`, bạn sẽ nhận được tập dữ liệu phẳng `{1, 2, 3, 4, 5, 6}`. Hơn nữa, phương thức `flat_map()` nhận một hàm làm đối số, cho phép bạn biến đổi mỗi tập dữ liệu trong tập dữ liệu lồng nhau trước khi làm phẳng. Ví dụ, nếu bạn truyền hàm `lambda ds: ds.batch(2)` cho `flat_map()`, thì nó sẽ biến đổi tập dữ liệu lồng nhau `{{1, 2}, {3, 4, 5, 6}}` thành tập dữ liệu phẳng `[[1, 2], [3, 4], [5, 6]]`: đó là một tập dữ liệu chứa 3 tensor, mỗi tensor có kích thước 2. Với những điều đó, chúng ta đã sẵn sàng làm phẳng tập dữ liệu của mình:

```
>>> dataset = tf.data.Dataset.range(6).window(4, shift=1, drop_remainder=True)
>>> dataset = dataset.flat_map(lambda window_dataset: window_dataset.batch(4))
>>> for window_tensor in dataset:
...     print(f"{window_tensor}")

[0 1 2 3]
[1 2 3 4]
[2 3 4 5]
```

Vì mỗi tập dữ liệu cửa sổ chứa chính xác bốn mục, việc gọi `batch(4)` trên một cửa sổ tạo ra một tensor duy nhất có kích thước 4. Tuyệt vời! Bây giờ chúng ta có một tập dữ liệu chứa các cửa sổ liên tiếp được biểu diễn dưới dạng tensor. Hãy tạo một hàm trợ giúp nhỏ để dễ dàng trích xuất các cửa sổ từ một tập dữ liệu hơn:

```
def to_windows(dataset, length):
    dataset = dataset.window(length, shift=1, drop_remainder=True)
```

```
return dataset.flat_map(lambda window_ds:
                        window_ds.batch(length))
```

Bước cuối cùng là chia mỗi cửa sổ thành đầu vào và mục tiêu, sử dụng phương thức `map()`. Chúng ta cũng có thể nhóm các cửa sổ kết quả thành các batch có kích thước 2:

```
>>> dataset = to_windows(tf.data.Dataset.range(6), 4) # 3 inputs + 1 target = 4
>>> dataset = dataset.map(lambda window: (window[:-1], window[-1]))
>>> list(dataset.batch(2))
[(<tf.Tensor: shape=(2, 3), dtype=int64, numpy=
  array([[0, 1, 2],
        [1, 2, 3]])>,
  <tf.Tensor: shape=(2,), dtype=int64, numpy=array([3, 4])>),
 (<tf.Tensor: shape=(1, 3), dtype=int64, numpy=array([[2, 3, 4]])>,
  <tf.Tensor: shape=(1,), dtype=int64, numpy=array([5])>)]
```

Như bạn có thể thấy, bây giờ chúng ta có cùng đầu ra như chúng ta đã nhận được trước đó với hàm `timeseries_dataset_from_array()` (với một chút nỗ lực hơn, nhưng sẽ sớm có giá trị). Bây giờ, trước khi chúng ta bắt đầu huấn luyện, chúng ta cần chia dữ liệu của mình thành một giai đoạn huấn luyện, một giai đoạn xác thực và một giai đoạn kiểm tra. Chúng ta sẽ tập trung vào lượng hành khách đường sắt trước. Chúng ta cũng sẽ chia tỷ lệ nó xuống một yếu tố một triệu, để đảm bảo các giá trị gần với phạm vi 0-1; điều này hoạt động tốt với khởi tạo trọng số và tốc độ học mặc định:

```
import pandas as pd
# Giả định df đã được tải và xử lý như trong phần trước
# (Nếu bạn chưa chạy phần đó, bạn cần chạy lại để có df)
# Ví dụ:
# from pathlib import Path
# path = Path("datasets/ridership/CTA_-_Ridership_-_Daily_Boarding_Totals.csv")
# df = pd.read_csv(path, parse_dates=["service_date"])
# df.columns = ["date", "day_type", "bus", "rail", "total"]
# df = df.sort_values("date").set_index("date")
# df = df.drop("total", axis=1)
# df = df.drop_duplicates()

rail_train = df["rail"]["2016-01":"2018-12"] / 1e6
rail_valid = df["rail"]["2019-01":"2019-05"] / 1e6
rail_test = df["rail"]["2019-06:"] / 1e6
```

Tiếp theo, hãy sử dụng `timeseries_dataset_from_array()` để tạo tập dữ liệu cho huấn luyện và xác thực. Vì gradient descent mong đợi các thể hiện trong tập huấn luyện là độc lập và phân bố giống hệt nhau (IID), như chúng ta đã thấy trong Chương 4, chúng ta phải đặt đối số `shuffle=True` để xáo trộn các cửa sổ huấn luyện (nhưng không phải nội dung của chúng):

```
seq_length = 56
train_ds = tf.keras.utils.timeseries_dataset_from_array(
```

```

    rail_train.to_numpy(),
    targets=rail_train[seq_length:],
    sequence_length=seq_length,
    batch_size=32,
    shuffle=True,
    seed=42
)
valid_ds = tf.keras.utils.timeseries_dataset_from_array(
    rail_valid.to_numpy(),
    targets=rail_valid[seq_length:],
    sequence_length=seq_length,
    batch_size=32
)

```

Và bây giờ chúng ta đã sẵn sàng xây dựng và huấn luyện bất kỳ mô hình hồi quy nào chúng ta muốn!

Dự báo bằng mô hình tuyến tính

Hãy thử một mô hình tuyến tính cơ bản trước. Chúng ta sẽ sử dụng hàm mất mát Huber, thường hoạt động tốt hơn là trực tiếp giảm thiểu MAE, như đã thảo luận trong Chương 10. Chúng ta cũng sẽ sử dụng early stopping:

```

tf.random.set_seed(42)
model = tf.keras.Sequential([
    tf.keras.layers.Dense(1, input_shape=[seq_length])
])

early_stopping_cb = tf.keras.callbacks.EarlyStopping(
    monitor="val_mae", patience=50, restore_best_weights=True)
opt = tf.keras.optimizers.SGD(learning_rate=0.02, momentum=0.9)
model.compile(loss=tf.keras.losses.Huber(), optimizer=opt, metrics=["mae"])
history = model.fit(train_ds, validation_data=valid_ds, epochs=500, callbacks
=[early_stopping_cb])

```

Mô hình này đạt MAE xác thực khoảng 37.866 (kết quả của bạn có thể khác). Điều này tốt hơn dự báo ngây thơ, nhưng tệ hơn mô hình SARIMA.

Chúng ta có thể làm tốt hơn với một RNN không? Hãy xem!

Dự báo bằng RNN đơn giản

Hãy thử RNN cơ bản nhất, chứa một lớp hồi quy đơn với chỉ một nơ-ron hồi quy, như chúng ta đã thấy trong Hình 15-1:

```

model = tf.keras.Sequential([
    tf.keras.layers.SimpleRNN(1, input_shape=[None, 1])
])

```

Tất cả các lớp hồi quy trong Keras mong đợi đầu vào 3D có hình dạng [batch size, time steps, dimensionality], trong đó dimensionality là 1 đối với chuỗi thời gian đơn biến và

lớn hơn đối với chuỗi thời gian đa biến. Nhớ lại rằng đối số `input_shape` bỏ qua chiều đầu tiên (tức là kích thước batch), và vì các lớp hồi quy có thể chấp nhận các chuỗi đầu vào có độ dài bất kỳ, chúng ta có thể đặt chiều thứ hai thành `None`, có nghĩa là “kích thước bất kỳ”. Cuối cùng, vì chúng ta đang xử lý một chuỗi thời gian đơn biến, chúng ta cần kích thước của chiều cuối cùng là 1. Đây là lý do tại sao chúng ta chỉ định hình dạng đầu vào `[None, 1]`: nó có nghĩa là “các chuỗi đơn biến có độ dài bất kỳ”. Lưu ý rằng các tập dữ liệu thực sự chứa đầu vào có hình dạng `[batch_size, time_steps]`, vì vậy chúng ta đang thiếu chiều cuối cùng, có kích thước 1, nhưng Keras đủ tốt bụng để thêm nó cho chúng ta trong trường hợp này.

Mô hình này hoạt động chính xác như chúng ta đã thấy trước đó: trạng thái ban đầu $h^{(init)}$ được đặt thành 0, và nó được truyền đến một nơ-ron hồi quy duy nhất, cùng với giá trị của bước thời gian đầu tiên, $x^{(0)}$. Nơ-ron tính toán tổng trọng số của các giá trị này cộng với số hạng độ lệch, và nó áp dụng hàm kích hoạt cho kết quả, sử dụng hàm hyperbolic tangent theo mặc định. Kết quả là đầu ra đầu tiên, $y^{(0)}$. Trong một RNN đơn giản, đầu ra này cũng là trạng thái mới $h^{(0)}$. Trạng thái mới này được truyền đến cùng một nơ-ron hồi quy cùng với giá trị đầu vào tiếp theo, $x^{(1)}$, và quá trình lặp lại cho đến bước thời gian cuối cùng. Cuối cùng, lớp chỉ xuất ra giá trị cuối cùng: trong trường hợp của chúng ta, các chuỗi dài 56 bước, vì vậy giá trị cuối cùng là $y^{(55)}$. Tất cả điều này được thực hiện đồng thời cho mỗi chuỗi trong batch, trong trường hợp này có 32 chuỗi.

Đó là mô hình hồi quy đầu tiên của chúng ta! Đó là một mô hình chuỗi-sang-vector. Vì chỉ có một nơ-ron đầu ra, vector đầu ra có kích thước là 1.

Bây giờ, nếu bạn biên dịch, huấn luyện và đánh giá mô hình này giống như mô hình trước, bạn sẽ thấy rằng nó không tốt chút nào: MAE xác thực của nó lớn hơn 100.000! Ôi. Điều đó đã được dự đoán trước, vì hai lý do:

12. Mô hình chỉ có một nơ-ron hồi quy duy nhất, vì vậy dữ liệu duy nhất nó có thể sử dụng để đưa ra dự đoán ở mỗi bước thời gian là giá trị đầu vào ở bước thời gian hiện tại và giá trị đầu ra từ bước thời gian trước đó. Điều đó không đủ để dựa vào! Nói cách khác, bộ nhớ của RNN cực kỳ hạn chế: nó chỉ là một số duy nhất, đầu ra trước đó của nó. Và hãy đếm xem mô hình này có bao nhiêu tham số: vì chỉ có một nơ-ron hồi quy với chỉ hai giá trị đầu vào, toàn bộ mô hình chỉ có ba tham số (hai trọng số cộng với một số hạng độ lệch). Điều đó còn xa mới đủ cho chuỗi thời gian này. Ngược lại, mô hình trước của chúng ta có thể xem xét tất cả 56 giá trị trước đó cùng một lúc, và nó có tổng cộng 57 tham số.
13. Chuỗi thời gian chứa các giá trị từ 0 đến khoảng 1.4, nhưng vì hàm kích hoạt mặc định là `tanh`, lớp hồi quy chỉ có thể xuất ra các giá trị từ -1 đến +1. Không có cách nào nó có thể dự đoán các giá trị từ 1.0 đến 1.4.

Hãy khắc phục cả hai vấn đề này: chúng ta sẽ tạo một mô hình với một lớp hồi quy lớn hơn, chứa 32 nơ-ron hồi quy, và chúng ta sẽ thêm một lớp đầu ra dày đặc lên trên nó với một nơ-ron đầu ra duy nhất và không có hàm kích hoạt. Lớp hồi quy sẽ có thể mang nhiều thông tin

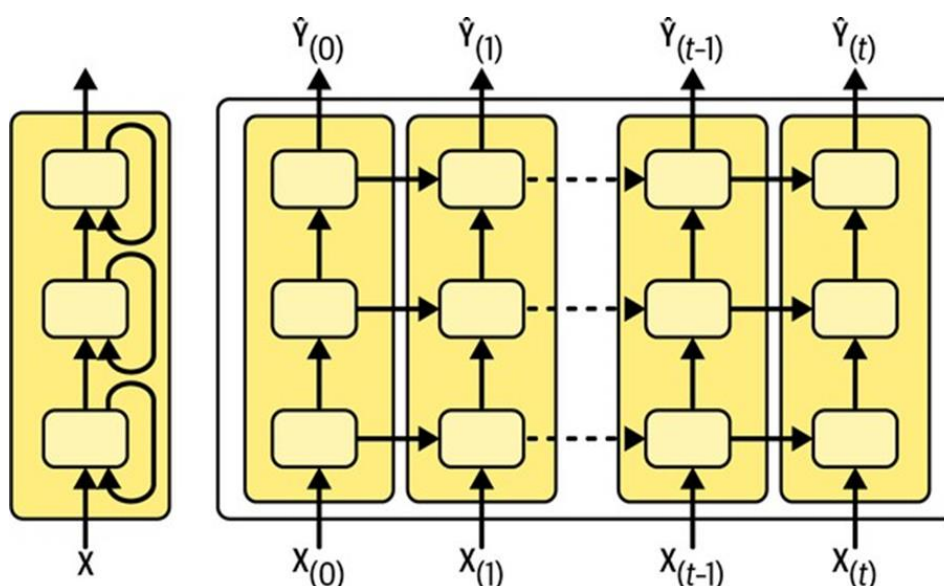
hơn từ bước thời gian này sang bước thời gian tiếp theo, và lớp đầu ra dày đặc sẽ chiếu đầu ra cuối cùng từ 32 chiều xuống 1, không có bất kỳ ràng buộc phạm vi giá trị nào:

```
univar_model = tf.keras.Sequential([
    tf.keras.layers.SimpleRNN(32, input_shape=[None, 1]),
    tf.keras.layers.Dense(1) # no activation function by default
])
```

Bây giờ nếu bạn biên dịch, điều chỉnh và đánh giá mô hình này giống như mô hình trước, bạn sẽ thấy rằng MAE xác thực của nó đạt 27.703. Đó là mô hình tốt nhất chúng ta đã huấn luyện cho đến nay, và nó thậm chí còn đánh bại mô hình SARIMA: chúng ta đang làm khá tốt!

Dự báo bằng RNN sâu

Thông thường, người ta thường chồng nhiều lớp ô lên nhau, như thể hiện trong Hình 15-10. Điều này tạo thành một **RNN sâu**.



Hình 15-10. Một RNN sâu (trái) được mở rộng theo thời gian (phải)

Việc triển khai một RNN sâu với Keras rất đơn giản: chỉ cần chồng các lớp hồi quy lên nhau. Trong ví dụ sau, chúng ta sử dụng ba lớp SimpleRNN (nhưng chúng ta có thể sử dụng bất kỳ loại lớp hồi quy nào khác thay thế, chẳng hạn như lớp LSTM hoặc lớp GRU, mà chúng ta sẽ thảo luận ngay sau đây). Hai lớp đầu tiên là các lớp chuỗi-sang-chuỗi, và lớp cuối cùng là một lớp chuỗi-sang-vector. Cuối cùng, lớp Dense tạo ra dự báo của mô hình (bạn có thể coi nó là một lớp vector-sang-vector). Vì vậy, mô hình này giống như mô hình được biểu diễn trong Hình 15-10, ngoại trừ các đầu ra $\hat{y}^{(0)}$ đến $\hat{y}^{(t-1)}$ bị bỏ qua, và có một lớp Dense nằm trên $\hat{y}^{(t)}$, xuất ra dự báo thực tế:

```
import tensorflow as tf
import pandas as pd
import numpy as np
```



```

# Giả định df đã được tải và xử lý như trong các phần trước
# (Nếu bạn chưa chạy phần đó, bạn cần chạy lại để có df)
# Ví dụ:
# from pathlib import Path
# path = Path("datasets/ridership/CTA_-_Ridership_-_Daily_Boarding_Totals.csv")
# df = pd.read_csv(path, parse_dates=["service_date"])
# df.columns = ["date", "day_type", "bus", "rail", "total"]
# df = df.sort_values("date").set_index("date")
# df = df.drop("total", axis=1)
# df = df.drop_duplicates()

# Chia dữ liệu và chuẩn hóa như đã làm trước đó
rail_train = df["rail"]["2016-01":"2018-12"] / 1e6
rail_valid = df["rail"]["2019-01":"2019-05"] / 1e6
rail_test = df["rail"]["2019-06:"] / 1e6

seq_length = 56
train_ds = tf.keras.utils.timeseries_dataset_from_array(
    rail_train.to_numpy(),
    targets=rail_train[seq_length:],
    sequence_length=seq_length,
    batch_size=32,
    shuffle=True,
    seed=42
)
valid_ds = tf.keras.utils.timeseries_dataset_from_array(
    rail_valid.to_numpy(),
    targets=rail_valid[seq_length:],
    sequence_length=seq_length,
    batch_size=32
)

deep_model = tf.keras.Sequential([
    tf.keras.layers.SimpleRNN(32, return_sequences=True,
                               input_shape=[None, 1]),
    tf.keras.layers.SimpleRNN(32, return_sequences=True),
    tf.keras.layers.SimpleRNN(32),
    tf.keras.layers.Dense(1)
])

# Biên dịch và huấn luyện mô hình (sử dụng cùng các đối số từ mô hình tuyến tính trước)
early_stopping_cb = tf.keras.callbacks.EarlyStopping(
    monitor="val_mae", patience=50, restore_best_weights=True)
opt = tf.keras.optimizers.SGD(learning_rate=0.02, momentum=0.9)
deep_model.compile(loss=tf.keras.losses.Huber(), optimizer=opt, metrics=["mae"])

```

```
history = deep_model.fit(train_ds, validation_data=valid_ds, epochs=500, call backs=[early_stopping_cb])
```

Nếu bạn huấn luyện và đánh giá mô hình này, bạn sẽ thấy rằng nó đạt MAE khoảng 31.211. Con số này tốt hơn cả hai đường cơ sở, nhưng nó không đánh bại RNN “nông” hơn của chúng ta. Có vẻ như RNN này hơi quá lớn so với nhiệm vụ của chúng ta.

Dự báo chuỗi thời gian đa biến

Một ưu điểm lớn của mạng thần kinh là tính linh hoạt của chúng: đặc biệt, chúng có thể xử lý các chuỗi thời gian đa biến mà hầu như không cần thay đổi kiến trúc. Ví dụ, hãy thử dự báo chuỗi thời gian đường sắt bằng cách sử dụng cả dữ liệu xe buýt và đường sắt làm đầu vào. Trên thực tế, hãy thêm cả loại ngày nữa! Vì chúng ta luôn có thể biết trước liệu ngày mai sẽ là ngày trong tuần, cuối tuần hay ngày lễ, chúng ta có thể dịch chuỗi loại ngày một ngày về phía trước, để mô hình được cung cấp loại ngày của ngày mai làm đầu vào. Để đơn giản, chúng ta sẽ thực hiện việc xử lý này bằng cách sử dụng Pandas:

```
# Giả định df đã được tải và xử lý như trong các phần trước
df_mulvar = df[["bus", "rail"]] / 1e6 # sử dụng cả chuỗi xe buýt & đường sắt làm đầu vào
df_mulvar["next_day_type"] = df["day_type"].shift(-1) # chúng ta biết loại ngày của ngày mai
df_mulvar = pd.get_dummies(df_mulvar) # mã hóa one-hot loại ngày
```

Bây giờ df_mulvar là một DataFrame với năm cột: dữ liệu xe buýt và đường sắt, cộng với ba cột chứa mã hóa one-hot của loại ngày tiếp theo (nhớ lại rằng có ba loại ngày có thể có, W, A và U). Tiếp theo, chúng ta có thể tiến hành như chúng ta đã làm trước đó. Đầu tiên chúng ta chia dữ liệu thành ba giai đoạn, để huấn luyện, xác thực và kiểm tra:

```
mulvar_train = df_mulvar["2016-01":"2018-12"]
mulvar_valid = df_mulvar["2019-01":"2019-05"]
mulvar_test = df_mulvar["2019-06":]
```

Sau đó, chúng ta tạo các tập dữ liệu:

```
seq_length = 56 # Vẫn sử dụng độ dài chuỗi 56
train_mulvar_ds = tf.keras.utils.timeseries_dataset_from_array(
    mulvar_train.to_numpy(), # sử dụng tất cả 5 cột làm đầu vào
    targets=mulvar_train["rail"][seq_length:], # chỉ dự báo chuỗi đường sắt
    sequence_length=seq_length,
    batch_size=32,
    shuffle=True,
    seed=42
)
valid_mulvar_ds = tf.keras.utils.timeseries_dataset_from_array(
    mulvar_valid.to_numpy(),
    targets=mulvar_valid["rail"][seq_length:],
    sequence_length=seq_length,
    batch_size=32
)
```

Và cuối cùng chúng ta tạo RNN:

```
mulvar_model = tf.keras.Sequential([
    tf.keras.layers.SimpleRNN(32, input_shape=[None, 5]),
    tf.keras.layers.Dense(1)
])

# Biên dịch và huấn luyện mô hình
mulvar_model.compile(loss=tf.keras.losses.Huber(), optimizer=opt, metrics=["mae"])
history_mulvar = mulvar_model.fit(train_mulvar_ds, validation_data=valid_mulvar_ds, epochs=500, callbacks=[early_stopping_cb])
```

Lưu ý rằng sự khác biệt duy nhất so với RNN univar_model chúng ta đã xây dựng trước đó là hình dạng đầu vào: ở mỗi bước thời gian, mô hình hiện nhận năm đầu vào thay vì một. Mô hình này thực sự đạt MAE xác thực là 22.062. Bây giờ chúng ta đang đạt được tiến bộ lớn!

Trên thực tế, không quá khó để làm cho RNN dự báo cả lượng hành khách xe buýt và đường sắt. Bạn chỉ cần thay đổi các mục tiêu khi tạo tập dữ liệu, đặt chúng thành mulvar_train[["bus", "rail"]][seq_length:] cho tập huấn luyện, và mulvar_valid[["bus", "rail"]][seq_length:] cho tập xác thực. Bạn cũng phải thêm một nơ-ron bổ sung vào lớp Dense đầu ra, vì giờ đây nó phải đưa ra hai dự báo: một cho lượng hành khách xe buýt vào ngày mai, và một cho đường sắt. Chỉ vậy thôi! Như chúng ta đã thảo luận trong Chương 10, việc sử dụng một mô hình duy nhất cho nhiều nhiệm vụ liên quan thường mang lại hiệu suất tốt hơn so với việc sử dụng một mô hình riêng biệt cho mỗi nhiệm vụ, vì các đặc trưng được học cho một nhiệm vụ có thể hữu ích cho các nhiệm vụ khác, và cũng vì việc phải thực hiện tốt trên nhiều nhiệm vụ ngăn mô hình bị quá khớp (đó là một dạng điều hòa). Tuy nhiên, điều đó phụ thuộc vào nhiệm vụ, và trong trường hợp cụ thể này, RNN đa nhiệm vụ dự báo cả lượng hành khách xe buýt và đường sắt không hoạt động tốt bằng các mô hình chuyên dụng dự báo một trong hai (sử dụng cả năm cột làm đầu vào). Tuy nhiên, nó đạt MAE xác thực là 25.330 cho đường sắt và 26.369 cho xe buýt, khá tốt.

Dự báo nhiều bước thời gian phía trước

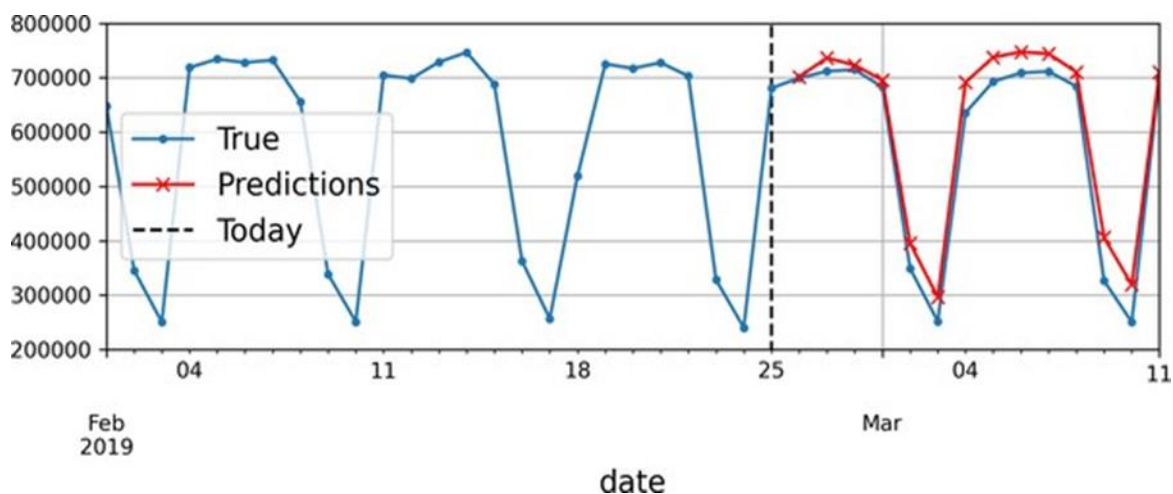
Cho đến nay, chúng ta chỉ dự đoán giá trị ở bước thời gian tiếp theo, nhưng chúng ta cũng có thể dễ dàng dự đoán giá trị nhiều bước phía trước bằng cách thay đổi các mục tiêu một cách thích hợp (ví dụ: để dự đoán lượng hành khách 2 tuần kể từ bây giờ, chúng ta có thể chỉ cần thay đổi các mục tiêu là giá trị 14 ngày tới thay vì 1 ngày tới). Nhưng điều gì sẽ xảy ra nếu chúng ta muốn dự đoán 14 giá trị tiếp theo? Tùy chọn đầu tiên là lấy RNN univar_model mà chúng ta đã huấn luyện trước đó cho chuỗi thời gian đường sắt, làm cho nó dự đoán giá trị tiếp theo, và thêm giá trị đó vào đầu vào, hành động như thể giá trị dự đoán đã thực sự xảy ra; sau đó chúng ta sẽ sử dụng lại mô hình để dự đoán giá trị tiếp theo, và cứ thế tiếp tục, như trong đoạn mã sau:

```
# Đảm bảo univar_model đã được huấn luyện
# tf.random.set_seed(42)
# univar_model = tf.keras.Sequential([
# tf.keras.layers.SimpleRNN(32, input_shape=[None,1]),
```

```
# tf.keras.layers.Dense(1)
# ])
# early_stopping_cb và opt đã được định nghĩa
# univar_model.compile(loss=tf.keras.losses.Huber(), optimizer=opt, metrics=
# ["mae"])
# history_univar = univar_model.fit(train_ds, validation_data=valid_ds, epoch
# s=500, callbacks=[early_stopping_cb])

X = rail_valid.to_numpy()[np.newaxis, :seq_length, np.newaxis]
for step_ahead in range(14):
    y_pred_one = univar_model.predict(X)
    X = np.concatenate([X, y_pred_one.reshape(1, 1, 1)], axis=1)
```

Trong mã này, chúng ta lấy lượng hành khách đường sắt của 56 ngày đầu tiên của giai đoạn xác thực, và chúng ta chuyển đổi dữ liệu thành một mảng NumPy có hình dạng [1, 56, 1] (nhớ lại rằng các lớp hồi quy mong đợi đầu vào 3D). Sau đó, chúng ta liên tục sử dụng mô hình để dự báo giá trị tiếp theo, và chúng ta nối mỗi dự báo vào chuỗi đầu vào, dọc theo trục thời gian (axis=1). Các dự báo kết quả được vẽ trong Hình 15-11.



Hình 15-11. Dự báo 14 bước phía trước, mỗi lần 1 bước

Tùy chọn thứ hai là huấn luyện một RNN để dự đoán 14 giá trị tiếp theo trong một lần. Chúng ta vẫn có thể sử dụng một mô hình chuỗi-sang-vector, nhưng nó sẽ xuất ra 14 giá trị thay vì 1. Tuy nhiên, trước tiên chúng ta cần thay đổi các mục tiêu thành các vector chứa 14 giá trị tiếp theo. Để làm điều này, chúng ta có thể sử dụng lại `timeseries_dataset_from_array()`, nhưng lần này yêu cầu nó tạo tập dữ liệu không có mục tiêu (`targets=None`) và với các chuỗi dài hơn, có độ dài `seq_length + 14`. Sau đó, chúng ta có thể sử dụng phương thức `map()` của tập dữ liệu để áp dụng một hàm tùy chỉnh cho mỗi batch của chuỗi, chia chúng thành đầu vào và mục tiêu. Trong ví dụ này, chúng ta sử dụng chuỗi thời gian đa biến làm đầu vào (sử dụng cả năm cột), và chúng ta dự báo lượng hành khách đường sắt trong 14 ngày tới:

```
def split_inputs_and_targets(mulvar_series, ahead=14, target_col=1):
    return mulvar_series[:, :-ahead], mulvar_series[:, -ahead:, target_col]
```

```

ahead_train_ds = tf.keras.utils.timeseries_dataset_from_array(
    mulvar_train.to_numpy(),
    targets=None,
    sequence_length=seq_length + 14,
    batch_size=32,
    shuffle=True,
    seed=42
).map(split_inputs_and_targets)

ahead_valid_ds = tf.keras.utils.timeseries_dataset_from_array(
    mulvar_valid.to_numpy(),
    targets=None,
    sequence_length=seq_length + 14,
    batch_size=32
).map(split_inputs_and_targets)

```

Bây giờ chúng ta chỉ cần lớp đầu ra có 14 đơn vị thay vì 1:

```

ahead_model = tf.keras.Sequential([
    tf.keras.layers.SimpleRNN(32, input_shape=[None, 5]),
    tf.keras.layers.Dense(14)
])

# Biên dịch và huấn luyện mô hình
ahead_model.compile(loss=tf.keras.losses.Huber(), optimizer=opt, metrics=["mae"])
history_ahead = ahead_model.fit(ahead_train_ds, validation_data=ahead_valid_ds, epochs=500, callbacks=[early_stopping_cb])

```

Sau khi huấn luyện mô hình này, bạn có thể dự đoán 14 giá trị tiếp theo cùng một lúc như sau:

```

X = mulvar_valid.to_numpy()[np.newaxis, :seq_length] # shape [1, 56, 5]
Y_pred = ahead_model.predict(X) # shape [1, 14]

```

Cách tiếp cận này hoạt động khá tốt. Các dự báo của nó cho ngày tiếp theo rõ ràng tốt hơn các dự báo của nó cho 14 ngày trong tương lai, nhưng nó không tích lũy lỗi như cách tiếp cận trước đó. Tuy nhiên, chúng ta vẫn có thể làm tốt hơn, sử dụng mô hình chuỗi-sang-chuỗi (hoặc seq2seq).

Dự báo bằng mô hình chuỗi-sang-chuỗi Thay vì huấn luyện mô hình để dự báo 14 giá trị tiếp theo chỉ ở bước thời gian cuối cùng, chúng ta có thể huấn luyện nó dự báo 14 giá trị tiếp theo ở mỗi và mọi bước thời gian. Nói cách khác, chúng ta có thể biến RNN chuỗi-sang-vector này thành một **RNN chuỗi-sang-chuỗi**. Ưu điểm của kỹ thuật này là hàm mất mát sẽ chứa một số hạng cho đầu ra của RNN ở mỗi và mọi bước thời gian, không chỉ cho đầu ra ở bước thời gian cuối cùng. Điều này có nghĩa là sẽ có nhiều gradient lỗi hơn chảy qua mô hình, và chúng sẽ không phải chảy qua thời gian nhiều vì chúng sẽ đến từ đầu ra của mỗi bước thời gian, không chỉ bước cuối cùng. Điều này sẽ vừa ổn định vừa tăng tốc độ huấn luyện. Để rõ ràng, tại bước thời gian 0, mô hình sẽ xuất ra một vector chứa các dự báo cho các bước thời

gian 1 đến 14, sau đó tại bước thời gian 1, mô hình sẽ dự báo các bước thời gian 2 đến 15, v.v. Nói cách khác, các mục tiêu là các chuỗi các cửa sổ liên tiếp, được dịch chuyển một bước thời gian ở mỗi bước thời gian. Mục tiêu không còn là một vector nữa, mà là một chuỗi có cùng độ dài với đầu vào, chứa một vector 14 chiều ở mỗi bước. Việc chuẩn bị tập dữ liệu không hề đơn giản, vì mỗi thể hiện có một cửa sổ làm đầu vào và một chuỗi các cửa sổ làm đầu ra. Một cách để làm điều này là sử dụng hàm tiện ích `to_windows()` mà chúng ta đã tạo trước đó, hai lần liên tiếp, để có được các cửa sổ của các cửa sổ liên tiếp. Ví dụ, hãy biến chuỗi các số từ 0 đến 6 thành một tập dữ liệu chứa các chuỗi gồm 4 cửa sổ liên tiếp, mỗi cửa sổ có độ dài 3:

```
import tensorflow as tf
import numpy as np

# Định nghĩa lại hàm to_windows nếu chưa có
def to_windows(dataset, length):
    dataset = dataset.window(length, shift=1, drop_remainder=True)
    return dataset.flat_map(lambda window_ds:
                           window_ds.batch(length))

my_series = tf.data.Dataset.range(7)
dataset = to_windows(to_windows(my_series, 3), 4)

for item in dataset:
    print(item)

<tf.Tensor: shape=(4, 3), dtype=int64, numpy=
array([[0, 1, 2],
       [1, 2, 3],
       [2, 3, 4],
       [3, 4, 5]])>
<tf.Tensor: shape=(4, 3), dtype=int64, numpy=
array([[1, 2, 3],
       [2, 3, 4],
       [3, 4, 5],
       [4, 5, 6]])>
```

Bây giờ chúng ta có thể sử dụng phương thức `map()` để chia các cửa sổ của các cửa sổ này thành đầu vào và mục tiêu:

```
dataset = dataset.map(lambda S: (S[:, 0], S[:, 1:]))

for item in dataset:
    print(item)

(<tf.Tensor: shape=(4,), dtype=int64, numpy=array([0, 1, 2, 3])>, <tf.Tensor:
shape=(4, 2), dtype=int64, numpy=
array([[1, 2],
       [2, 3],
       [3, 4],
```

```

[4, 5]])>)
(<tf.Tensor: shape=(4,), dtype=int64, numpy=array([1, 2, 3, 4])>, <tf.Tensor:
shape=(4, 2), dtype=int64, numpy=
array([[2, 3],
       [3, 4],
       [4, 5],
       [5, 6]])>)

```

Bây giờ tập dữ liệu chứa các chuỗi có độ dài 4 làm đầu vào, và các mục tiêu là các chuỗi chứa hai bước tiếp theo, cho mỗi bước thời gian. Ví dụ, chuỗi đầu vào đầu tiên là [0, 1, 2, 3], và các mục tiêu tương ứng của nó là [[1, 2], [2, 3], [3, 4], [4, 5]], đó là hai giá trị tiếp theo cho mỗi bước thời gian. Nếu bạn giống tôi, bạn có thể sẽ cần vài phút để hiểu rõ điều này. Cứ từ từ!

Hãy tạo một hàm tiện ích nhỏ khác để chuẩn bị tập dữ liệu cho mô hình chuỗi-sang-chuỗi của chúng ta. Nó cũng sẽ lo việc xáo trộn (tùy chọn) và nhóm batch:

```

import pandas as pd # Cần thiết cho mulvar_train, mulvar_valid

# Giả định df đã được tải và xử lý như trong các phần trước
# Ví dụ:
# from pathlib import Path
# path = Path("datasets/ridership/CTA_-_Ridership_-_Daily_Boarding_Totals.csv")
# df = pd.read_csv(path, parse_dates=["service_date"])
# df.columns = ["date", "day_type", "bus", "rail", "total"]
# df = df.sort_values("date").set_index("date")
# df = df.drop("total", axis=1)
# df = df.drop_duplicates()

# Tạo df_mulvar và chia dữ liệu như đã làm trước đó
df_mulvar = df[["bus", "rail"]] / 1e6
df_mulvar["next_day_type"] = df["day_type"].shift(-1)
df_mulvar = pd.get_dummies(df_mulvar)

mulvar_train = df_mulvar["2016-01":"2018-12"]
mulvar_valid = df_mulvar["2019-01":"2019-05"]
mulvar_test = df_mulvar["2019-06":]

def to_seq2seq_dataset(series, seq_length=56, ahead=14, target_col=1,
                       batch_size=32, shuffle=False, seed=None):
    ds = to_windows(tf.data.Dataset.from_tensor_slices(series),
                    ahead + 1)
    ds = to_windows(ds, seq_length).map(lambda S: (S[:, 0], S[:, 1:, target_col])) # Sử dụng target_col để lấy cột mục tiêu

    if shuffle:
        ds = ds.shuffle(8 * batch_size, seed=seed)
    return ds.batch(batch_size)

```


Bây giờ chúng ta có thể sử dụng hàm này để tạo tập dữ liệu:

```
seq2seq_train = to_seq2seq_dataset(mulvar_train, shuffle=True, seed=42)
seq2seq_valid = to_seq2seq_dataset(mulvar_valid)
```

Và cuối cùng, chúng ta có thể xây dựng mô hình chuỗi-sang-chuỗi:

```
seq2seq_model = tf.keras.Sequential([
    tf.keras.layers.SimpleRNN(32, return_sequences=True,
                               input_shape=[None, 5]),
    tf.keras.layers.Dense(14) # output 14 values
])

# Biên dịch và huấn luyện mô hình (sử dụng các đối số tương tự)
early_stopping_cb = tf.keras.callbacks.EarlyStopping(
    monitor="val_mae", patience=50, restore_best_weights=True)
opt = tf.keras.optimizers.SGD(learning_rate=0.02, momentum=0.9)
seq2seq_model.compile(loss=tf.keras.losses.Huber(), optimizer=opt, metrics=["mae"])
history_seq2seq = seq2seq_model.fit(seq2seq_train, validation_data=(seq2seq_valid, seq2seq_valid), epochs=500, callbacks=[early_stopping_cb])
```

Nó gần như giống hệt mô hình trước của chúng ta: điểm khác biệt duy nhất là chúng ta đặt `return_sequences=True` trong lớp `SimpleRNN`. Bằng cách này, nó sẽ xuất ra một chuỗi các vector (mỗi vector có kích thước 32), thay vì xuất ra một vector duy nhất ở bước thời gian cuối cùng. Lớp `Dense` đủ thông minh để xử lý các chuỗi làm đầu vào: nó sẽ được áp dụng ở mỗi bước thời gian, lấy một vector 32 chiều làm đầu vào và xuất ra một vector 14 chiều. Trên thực tế, một cách khác để có được kết quả chính xác tương tự là sử dụng một lớp `Conv1D` với kích thước kernel là 1: `Conv1D(14, kernel_size=1)`.

Mã huấn luyện vẫn như thường lệ. Trong quá trình huấn luyện, tất cả các đầu ra của mô hình đều được sử dụng, nhưng sau khi huấn luyện, chỉ có đầu ra của bước thời gian cuối cùng là quan trọng, và phần còn lại có thể bị bỏ qua. Ví dụ, chúng ta có thể dự báo lượng hành khách đường sắt trong 14 ngày tới như sau:

```
seq_length = 56 # Đảm bảo seq_length được định nghĩa
X = mulvar_valid.to_numpy()[np.newaxis, :seq_length]
y_pred_14 = seq2seq_model.predict(X)[0, -1] # chỉ lấy đầu ra của bước thời gian cuối cùng
```

Nếu bạn đánh giá dự báo của mô hình này cho $t + 1$, bạn sẽ thấy MAE thực là 25.519. Đối với $t + 2$ là 26.274, và hiệu suất tiếp tục giảm dần khi mô hình cố gắng dự báo xa hơn trong tương lai. Tại $t + 14$, MAE là 34.322.

Các RNN đơn giản có thể khá tốt trong việc dự báo chuỗi thời gian hoặc xử lý các loại chuỗi khác, nhưng chúng không hoạt động tốt trên các chuỗi hoặc chuỗi thời gian dài. Hãy thảo luận lý do và xem chúng ta có thể làm gì để khắc phục.

Dự báo bằng mô hình Sequence-to-Sequence

Thay vì huấn luyện mô hình chỉ dự báo 14 giá trị tiếp theo tại bước thời gian cuối cùng, chúng ta có thể huấn luyện nó dự báo 14 giá trị tiếp theo tại **mỗi và mọi bước thời gian**. Nói cách khác, chúng ta có thể biến RNN sequence-to-vector này thành **RNN sequence-to-sequence**. Ưu điểm của kỹ thuật này là hàm mất mát sẽ chứa một thành phần cho đầu ra của RNN tại mỗi và mọi bước thời gian, không chỉ cho đầu ra ở bước thời gian cuối cùng. Điều này có nghĩa là sẽ có nhiều gradient lỗi hơn truyền qua mô hình, và chúng sẽ không phải truyền qua thời gian nhiều vì chúng đến từ đầu ra của mỗi bước thời gian, không chỉ từ bước cuối cùng. Điều này sẽ giúp **ổn định và tăng tốc độ huấn luyện**.

Để rõ ràng hơn, tại bước thời gian 0, mô hình sẽ xuất ra một vector chứa các dự báo cho các bước thời gian từ 1 đến 14, sau đó tại bước thời gian 1, mô hình sẽ dự báo cho các bước thời gian từ 2 đến 15, v.v. Nói cách khác, các mục tiêu là các chuỗi cửa sổ liên tiếp, được dịch chuyển đi một bước thời gian tại mỗi bước thời gian. Mục tiêu không còn là một vector nữa, mà là một chuỗi có cùng độ dài với đầu vào, chứa một vector 14 chiều ở mỗi bước.

Việc chuẩn bị các tập dữ liệu không hề đơn giản, vì mỗi mẫu có một cửa sổ làm đầu vào và một chuỗi các cửa sổ làm đầu ra. Một cách để làm điều này là sử dụng hàm tiện ích `to_windows()` mà chúng ta đã tạo trước đó, hai lần liên tiếp, để nhận được các cửa sổ của các cửa sổ liên tiếp. Ví dụ, hãy biến chuỗi số từ 0 đến 6 thành một tập dữ liệu chứa các chuỗi gồm 4 cửa sổ liên tiếp, mỗi cửa sổ có độ dài 3:

```
>>> my_series = tf.data.Dataset.range(7)
>>> dataset = to_windows(to_windows(my_series, 3), 4)
>>> list(dataset)
[<tf.Tensor: shape=(4, 3), dtype=int64, numpy=
  array([[0, 1, 2],
        [1, 2, 3],
        [2, 3, 4],
        [3, 4, 5]])>,
 <tf.Tensor: shape=(4, 3), dtype=int64, numpy=
  array([[1, 2, 3],
        [2, 3, 4],
        [3, 4, 5],
        [4, 5, 6]])>]
```

Bây giờ chúng ta có thể sử dụng phương thức `map()` để tách các cửa sổ của cửa sổ này thành đầu vào và mục tiêu:

```
>>> dataset = dataset.map(lambda S: (S[:, 0], S[:, 1:]))
>>> list(dataset)
[(<tf.Tensor: shape=(4,), dtype=int64, numpy=array([0, 1, 2, 3])>,
 <tf.Tensor: shape=(4, 2), dtype=int64, numpy=
  array([[1, 2],
        [2, 3],
        [3, 4],
        [4, 5]])>),
 (<tf.Tensor: shape=(4,), dtype=int64, numpy=array([1, 2, 3, 4])>,
```

```
<tf.Tensor: shape=(4, 2), dtype=int64, numpy=
  array([[2, 3],
         [3, 4],
         [4, 5],
         [5, 6]])>)]
```

Bây giờ tập dữ liệu chứa các chuỗi có độ dài 4 làm đầu vào, và các mục tiêu là các chuỗi chứa hai bước tiếp theo, cho mỗi bước thời gian. Ví dụ, chuỗi đầu vào đầu tiên là [0, 1, 2, 3], và các mục tiêu tương ứng của nó là [[1, 2], [2, 3], [3, 4], [4, 5]], là hai giá trị tiếp theo cho mỗi bước thời gian. Nếu bạn giống tôi, bạn có thể sẽ cần vài phút để hiểu rõ điều này. Cứ từ từ nhé!

Hãy tạo một hàm tiện ích nhỏ khác để chuẩn bị các tập dữ liệu cho mô hình sequence-to-sequence của chúng ta. Nó cũng sẽ xử lý việc xáo trộn (tùy chọn) và tạo lô:

```
def to_seq2seq_dataset(series, seq_length=56, ahead=14, target_col=1,
                        batch_size=32, shuffle=False, seed=None):
    ds = to_windows(tf.data.Dataset.from_tensor_slices(series), ahead + 1)
    ds = to_windows(ds, seq_length).map(lambda S: (S[:, 0], S[:, 1:, 1]))
    if shuffle:
        ds = ds.shuffle(8 * batch_size, seed=seed)
    return ds.batch(batch_size)
```

Bây giờ chúng ta có thể sử dụng hàm này để tạo các tập dữ liệu:

```
seq2seq_train = to_seq2seq_dataset(mulvar_train, shuffle=True, seed=42)
seq2seq_valid = to_seq2seq_dataset(mulvar_valid)
```

Và cuối cùng, chúng ta có thể xây dựng mô hình sequence-to-sequence:

```
seq2seq_model = tf.keras.Sequential([
    tf.keras.layers.SimpleRNN(32, return_sequences=True, input_shape=[None, 5]
)],
    tf.keras.layers.Dense(14)
])
```

Nó gần như giống hệt với mô hình trước của chúng ta: điểm khác biệt duy nhất là chúng ta đặt `return_sequences=True` trong lớp `SimpleRNN`. Bằng cách này, nó sẽ xuất ra một chuỗi các vector (mỗi vector có kích thước 32), thay vì xuất ra một vector duy nhất ở bước thời gian cuối cùng. Lớp `Dense` đủ thông minh để xử lý các chuỗi làm đầu vào: nó sẽ được áp dụng tại mỗi bước thời gian, nhận một vector 32 chiều làm đầu vào và xuất ra một vector 14 chiều. Trên thực tế, một cách khác để có được kết quả hoàn toàn giống hệt là sử dụng một lớp `Conv1D` với kích thước kernel là 1: `Conv1D(14, kernel_size=1)`.

Mã huấn luyện vẫn như thường lệ. Trong quá trình huấn luyện, tất cả các đầu ra của mô hình đều được sử dụng, nhưng sau khi huấn luyện, chỉ có đầu ra của bước thời gian cuối cùng mới quan trọng, và phần còn lại có thể được bỏ qua. Ví dụ, chúng ta có thể dự báo lượng hành khách đi tàu cho 14 ngày tiếp theo như sau:

```
X = mulvar_valid.to_numpy()[np.newaxis, :seq_length]
y_pred_14 = seq2seq_model.predict(X)[0, -1] # chỉ lấy đầu ra của bước thời gi
an cuối cùng
```

Nếu bạn đánh giá các dự báo của mô hình này cho $t+1$, bạn sẽ tìm thấy một MAE (Sai số tuyệt đối trung bình) trên tập kiểm định là 25,519. Đối với $t+2$, nó là 26,274, và hiệu suất tiếp tục giảm dần khi mô hình cố gắng dự báo xa hơn trong tương lai. Tại $t+14$, MAE là 34,322.

Các RNN đơn giản có thể khá tốt trong việc dự báo chuỗi thời gian hoặc xử lý các loại chuỗi khác, nhưng chúng không hoạt động tốt trên các chuỗi thời gian hoặc chuỗi dài. Hãy thảo luận tại sao và xem chúng ta có thể làm gì về nó.

Xử lý các chuỗi dài

Để huấn luyện một RNN trên các chuỗi dài, chúng ta phải chạy nó qua nhiều bước thời gian, làm cho RNN được mở rộng trở thành một mạng rất sâu. Giống như bất kỳ mạng thần kinh sâu nào, nó có thể gặp phải vấn đề gradient không ổn định, được thảo luận trong Chương 11: nó có thể mất mãi mãi để huấn luyện, hoặc quá trình huấn luyện có thể không ổn định. Hơn nữa, khi một RNN xử lý một chuỗi dài, nó sẽ dần dần quên đi các đầu vào đầu tiên trong chuỗi. Hãy xem xét cả hai vấn đề này, bắt đầu với vấn đề gradient không ổn định.

Giải quyết vấn đề gradient không ổn định

Nhiều thủ thuật chúng ta đã sử dụng trong các mạng sâu để giảm bớt vấn đề gradient không ổn định cũng có thể được sử dụng cho RNN: khởi tạo tham số tốt, bộ tối ưu hóa nhanh hơn, dropout, v.v. Tuy nhiên, các hàm kích hoạt không bão hòa (ví dụ: ReLU) có thể không giúp ích nhiều ở đây. Trên thực tế, chúng thực sự có thể khiến RNN không ổn định hơn trong quá trình huấn luyện. Tại sao? Chà, giả sử gradient descent cập nhật các trọng số theo cách làm tăng nhẹ các đầu ra ở bước thời gian đầu tiên. Vì cùng các trọng số được sử dụng ở mọi bước thời gian, các đầu ra ở bước thời gian thứ hai cũng có thể tăng nhẹ, và những thứ ở bước thứ ba, v.v. cho đến khi các đầu ra bùng nổ—và một hàm kích hoạt không bão hòa không ngăn được điều đó. Bạn có thể giảm thiểu rủi ro này bằng cách sử dụng tốc độ học nhỏ hơn, hoặc bạn có thể sử dụng một hàm kích hoạt bão hòa như hàm hyperbolic tangent (điều này giải thích tại sao nó là mặc định). Cũng theo cách tương tự, các gradient có thể bùng nổ. Nếu bạn nhận thấy quá trình huấn luyện không ổn định, bạn có thể muốn theo dõi kích thước của các gradient (ví dụ: sử dụng TensorBoard) và có thể sử dụng **cắt gradient (gradient clipping)**. Hơn nữa, **chuẩn hóa hàng loạt (batch normalization)** không thể được sử dụng hiệu quả với RNN như với các mạng truyền thẳng sâu. Trên thực tế, bạn không thể sử dụng nó giữa các bước thời gian, chỉ giữa các lớp hồi quy. Để chính xác hơn, về mặt kỹ thuật, có thể thêm một lớp BN vào một ô bộ nhớ (như bạn sẽ thấy ngay sau đây) để nó sẽ được áp dụng ở mỗi bước thời gian (cả trên đầu vào cho bước thời gian đó và trên trạng thái ẩn từ bước trước). Tuy nhiên, cùng một lớp BN sẽ được sử dụng ở mỗi bước thời gian, với cùng các tham số, bất kể thang đo và độ lệch thực tế của đầu vào và trạng thái ẩn. Trong thực tế, điều này không mang lại kết quả tốt, như đã được César Laurent et al. chứng minh trong một bài báo năm 2015: các tác giả nhận thấy rằng BN chỉ có lợi một chút khi nó được áp dụng cho đầu vào của lớp, không phải cho trạng thái ẩn. Nói cách khác, nó tốt hơn một chút so với không có gì khi được

áp dụng giữa các lớp hồi quy (tức là theo chiều dọc trong Hình 15-10), nhưng không phải trong các lớp hồi quy (tức là theo chiều ngang). Trong Keras, bạn có thể áp dụng BN giữa các lớp đơn giản bằng cách thêm một lớp BatchNormalization trước mỗi lớp hồi quy, nhưng nó sẽ làm chậm quá trình huấn luyện và có thể không giúp ích nhiều.

Một dạng chuẩn hóa khác thường hoạt động tốt hơn với RNN: **chuẩn hóa lớp (layer normalization)**. Ý tưởng này được Jimmy Lei Ba et al. giới thiệu trong một bài báo năm 2016: nó rất giống với chuẩn hóa hàng loạt, nhưng thay vì chuẩn hóa trên chiều batch, chuẩn hóa lớp chuẩn hóa trên chiều đặc trưng. Một ưu điểm là nó có thể tính toán các thống kê cần thiết một cách nhanh chóng, ở mỗi bước thời gian, độc lập cho mỗi thể hiện. Điều này cũng có nghĩa là nó hoạt động theo cùng một cách trong quá trình huấn luyện và kiểm tra (ngược lại với BN), và nó không cần sử dụng trung bình động hàm mũ để ước tính các thống kê đặc trưng trên tất cả các thể hiện trong tập huấn luyện, như BN làm. Giống như BN, chuẩn hóa lớp học một tham số tỷ lệ và một tham số bù cho mỗi đầu vào. Trong một RNN, nó thường được sử dụng ngay sau khi kết hợp tuyến tính các đầu vào và các trạng thái ẩn. Hãy sử dụng Keras để triển khai chuẩn hóa lớp trong một ô bộ nhớ đơn giản. Để làm điều này, chúng ta cần định nghĩa một ô bộ nhớ tùy chỉnh, giống như một lớp thông thường, ngoại trừ phương thức `call()` của nó nhận hai đối số: các đầu vào ở bước thời gian hiện tại và các trạng thái ẩn từ bước thời gian trước đó. Lưu ý rằng đối số `states` là một danh sách chứa một hoặc nhiều tensor. Trong trường hợp một ô RNN đơn giản, nó chứa một tensor duy nhất bằng với đầu ra của bước thời gian trước đó, nhưng các ô khác có thể có nhiều tensor trạng thái (ví dụ: một LSTMCell có trạng thái dài hạn và trạng thái ngắn hạn, như bạn sẽ thấy ngay sau đây). Một ô cũng phải có thuộc tính `state_size` và thuộc tính `output_size`. Trong một RNN đơn giản, cả hai đều đơn giản bằng số đơn vị. Mã sau đây triển khai một ô bộ nhớ tùy chỉnh sẽ hoạt động như một SimpleRNNCell, ngoại trừ nó cũng sẽ áp dụng chuẩn hóa lớp ở mỗi bước thời gian:

```
class LNSimpleRNNCell(tf.keras.layers.Layer):
    def __init__(self, units, activation="tanh", **kwargs):
        super().__init__(**kwargs)
        self.state_size = units
        self.output_size = units
        self.simple_rnn_cell = tf.keras.layers.SimpleRNNCell(units,
                                                                activation=None)

        self.layer_norm = tf.keras.layers.LayerNormalization()
        self.activation = tf.keras.activations.get(activation)

    def call(self, inputs, states):
        outputs, new_states = self.simple_rnn_cell(inputs, states)
        norm_outputs = self.activation(self.layer_norm(outputs))
        return norm_outputs, [norm_outputs]
```

Hãy đi qua mã này:

- Lớp LNSimpleRNNCell của chúng ta kế thừa từ lớp `tf.keras.layers.Layer`, giống như bất kỳ lớp tùy chỉnh nào khác.

- Hàm tạo lấy số đơn vị và hàm kích hoạt mong muốn và đặt các thuộc tính `state_size` và `output_size`, sau đó tạo một `SimpleRNNCell` không có hàm kích hoạt (vì chúng ta muốn thực hiện chuẩn hóa lớp sau phép toán tuyến tính nhưng trước hàm kích hoạt). Sau đó, hàm tạo tạo lớp `LayerNormalization`, và cuối cùng nó tìm nạp hàm kích hoạt mong muốn.
- Phương thức `call()` bắt đầu bằng cách áp dụng `simple_rnn_cell`, tính toán sự kết hợp tuyến tính của các đầu vào hiện tại và các trạng thái ẩn trước đó, và nó trả về kết quả hai lần (thực ra, trong một `SimpleRNNCell`, các đầu ra chỉ bằng với các trạng thái ẩn: nói cách khác, `new_states[0]` bằng với `outputs`, vì vậy chúng ta có thể bỏ qua `new_states` một cách an toàn trong phần còn lại của phương thức `call()`).
- Tiếp theo, phương thức `call()` áp dụng chuẩn hóa lớp, sau đó là hàm kích hoạt. Cuối cùng, nó trả về các đầu ra hai lần: một lần dưới dạng đầu ra, và một lần dưới dạng các trạng thái ẩn mới. Để sử dụng ô tùy chỉnh này, tất cả những gì chúng ta cần làm là tạo một lớp `tf.keras.layers.RNN`, truyền cho nó một thể hiện ô:

```
custom_ln_model = tf.keras.Sequential([
    tf.keras.layers.RNN(LNSimpleRNNCell(32), return_sequences=True,
                        input_shape=[None, 5]),
    tf.keras.layers.Dense(14)
])

# Biên dịch và huấn luyện mô hình (sử dụng các đối số tương tự)
custom_ln_model.compile(loss=tf.keras.losses.Huber(), optimizer=opt, metrics=
["mae"])
history_custom_ln = custom_ln_model.fit(seq2seq_train, validation_data=seq2seq_val, epochs=500, callbacks=[early_stopping_cb])
```

Tương tự, bạn có thể tạo một ô tùy chỉnh để áp dụng dropout giữa mỗi bước thời gian. Nhưng có một cách đơn giản hơn: hầu hết các lớp và ô hồi quy được cung cấp bởi Keras có các siêu tham số dropout và `recurrent_dropout`: cái trước định nghĩa tỷ lệ dropout để áp dụng cho đầu vào, và cái sau định nghĩa tỷ lệ dropout cho các trạng thái ẩn, giữa các bước thời gian. Vì vậy, không cần thiết phải tạo một ô tùy chỉnh để áp dụng dropout ở mỗi bước thời gian trong một RNN. Với các kỹ thuật này, bạn có thể giảm thiểu vấn đề gradient không ổn định và huấn luyện một RNN hiệu quả hơn nhiều. Bây giờ chúng ta hãy xem cách giải quyết vấn đề bộ nhớ ngắn hạn.

Nghĩ lại vấn đề bộ nhớ ngắn hạn

Do các biến đổi mà dữ liệu trải qua khi đi qua một RNN, một số thông tin bị mất ở mỗi bước thời gian. Sau một thời gian, trạng thái của RNN hầu như không còn dấu vết nào của các đầu vào đầu tiên. Điều này có thể là một trở ngại lớn. Hãy tưởng tượng Dory con cá đang cố gắng dịch một câu dài; đến khi cô ấy đọc xong, cô ấy không biết câu đó bắt đầu như thế nào. Để giải quyết vấn đề này, nhiều loại ô có bộ nhớ dài hạn đã được giới thiệu. Chúng đã chứng minh thành công đến mức các ô cơ bản không còn được sử dụng nhiều nữa. Đầu tiên hãy cùng xem xét ô bộ nhớ dài hạn phổ biến nhất trong số này: ô LSTM.

Các ô LSTM Ô bộ nhớ dài hạn ngắn hạn (LSTM) được Sepp Hochreiter và Jürgen Schmidhuber đề xuất vào năm 1997 và dần được cải thiện qua nhiều năm bởi một số nhà nghiên cứu, chẳng hạn như Alex Graves, Haşim Sak, và Wojciech Zaremba. Nếu bạn coi ô LSTM như một hộp đen, nó có thể được sử dụng rất giống một ô cơ bản, ngoại trừ nó sẽ hoạt động tốt hơn nhiều; quá trình huấn luyện sẽ hội tụ nhanh hơn và nó sẽ phát hiện các mẫu dài hạn hơn trong dữ liệu. Trong Keras, bạn có thể đơn giản sử dụng lớp LSTM thay vì lớp SimpleRNN:

```
import tensorflow as tf
import pandas as pd
import numpy as np

# Giả định df đã được tải và xử lý như trong các phần trước
# (Nếu bạn chưa chạy phần đó, bạn cần chạy lại để có df)
# Ví dụ:
# from pathlib import Path
# path = Path("datasets/ridership/CTA_-_Ridership_-_Daily_Boarding_Totals.csv")
# df = pd.read_csv(path, parse_dates=["service_date"])
# df.columns = ["date", "day_type", "bus", "rail", "total"]
# df = df.sort_values("date").set_index("date")
# df = df.drop("total", axis=1)
# df = df.drop_duplicates()

# Tạo df_mulvar và chia dữ liệu như đã làm trước đó
df_mulvar = df[["bus", "rail"]] / 1e6
df_mulvar["next_day_type"] = df["day_type"].shift(-1)
df_mulvar = pd.get_dummies(df_mulvar)

mulvar_train = df_mulvar["2016-01":"2018-12"]
mulvar_valid = df_mulvar["2019-01":"2019-05"]
mulvar_test = df_mulvar["2019-06":]

seq_length = 56 # Vẫn sử dụng độ dài chuỗi 56
ahead = 14 # Số bước thời gian dự báo

def to_seq2seq_dataset(series, seq_length=56, ahead=14, target_col=1,
                        batch_size=32, shuffle=False, seed=None):
    ds = to_windows(tf.data.Dataset.from_tensor_slices(series),
                    ahead + 1)
    ds = to_windows(ds, seq_length).map(lambda S: (S[:, 0], S[:, 1:, target_col]))

    if shuffle:
        ds = ds.shuffle(8 * batch_size, seed=seed)
    return ds.batch(batch_size)

seq2seq_train = to_seq2seq_dataset(mulvar_train, shuffle=True, seed=42)
```

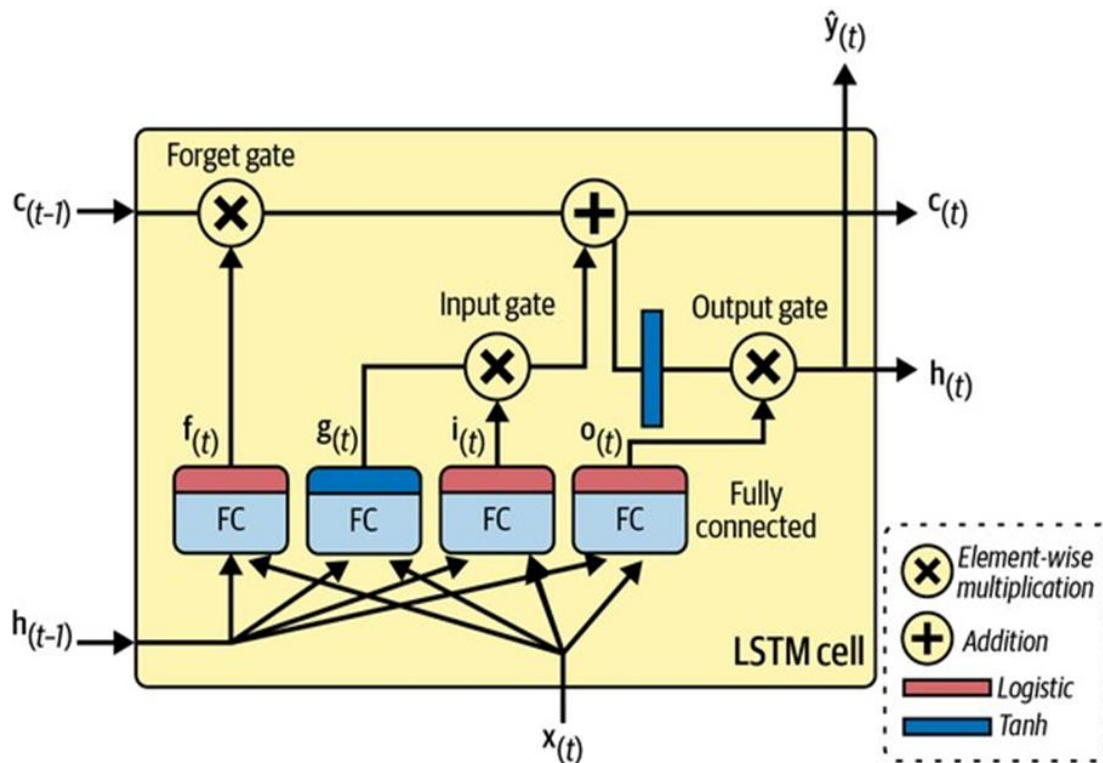
```
seq2seq_valid = to_seq2seq_dataset(mulvar_valid)

model = tf.keras.Sequential([
    tf.keras.layers.LSTM(32, return_sequences=True, input_shape=[None, 5]),
    tf.keras.layers.Dense(14)
])

# Biên dịch và huấn luyện mô hình (sử dụng các đối số tương tự)
early_stopping_cb = tf.keras.callbacks.EarlyStopping(
    monitor="val_mae", patience=50, restore_best_weights=True)
opt = tf.keras.optimizers.SGD(learning_rate=0.02, momentum=0.9)
model.compile(loss=tf.keras.losses.Huber(), optimizer=opt, metrics=["mae"])
history_lstm = model.fit(seq2seq_train, validation_data=seq2seq_valid, epochs=500, callbacks=[early_stopping_cb])
```

Ngoài ra, bạn có thể sử dụng lớp `tf.keras.layers.RNN` đa năng, truyền cho nó một `LSTMCell` làm đối số. Tuy nhiên, lớp LSTM sử dụng một triển khai được tối ưu hóa khi chạy trên GPU (xem Chương 19), vì vậy nói chung nên ưu tiên sử dụng nó (lớp RNN chủ yếu hữu ích khi bạn định nghĩa các ô tùy chỉnh, như chúng ta đã làm trước đó).

Vậy một ô LSTM hoạt động như thế nào? Kiến trúc của nó được thể hiện trong Hình 15-12. Nếu bạn không nhìn vào bên trong hộp, ô LSTM trông giống hệt một ô thông thường, ngoại trừ trạng thái của nó được chia thành hai vector: $h^{(t)}$ và $c^{(t)}$ (“c” là viết tắt của “cell”). Bạn có thể coi $h^{(t)}$ là trạng thái ngắn hạn và $c^{(t)}$ là trạng thái dài hạn.



Hình 15-12. Một ô LSTM

Bây giờ hãy mở hộp ra! Ý tưởng chính là mạng có thể học cách lưu trữ những gì vào trạng thái dài hạn, những gì cần loại bỏ và những gì cần đọc từ đó. Khi trạng thái dài hạn $c^{(t-1)}$ đi qua mạng từ trái sang phải, bạn có thể thấy rằng nó đi qua một **cổng quên (forget gate)**, loại bỏ một số ký ức, và sau đó nó thêm một số ký ức mới thông qua phép toán cộng (thêm các ký ức đã được chọn bởi một **cổng đầu vào (input gate)**). Kết quả $c^{(t)}$ được gửi thẳng ra ngoài, không có bất kỳ biến đổi nào nữa. Vì vậy, ở mỗi bước thời gian, một số ký ức bị loại bỏ và một số ký ức được thêm vào. Hơn nữa, sau phép toán cộng, trạng thái dài hạn được sao chép và truyền qua hàm tanh, và sau đó kết quả được lọc bởi **cổng đầu ra (output gate)**. Điều này tạo ra trạng thái ngắn hạn $h^{(t)}$ (bằng với đầu ra của ô cho bước thời gian này, $y^{(t)}$). Bây giờ hãy xem các ký ức mới đến từ đâu và các cổng hoạt động như thế nào.

Đầu tiên, vector đầu vào hiện tại $x^{(t)}$ và trạng thái ngắn hạn trước đó $h^{(t-1)}$ được đưa vào bốn lớp kết nối đầy đủ khác nhau. Tất cả chúng đều phục vụ một mục đích khác nhau:

- **Lớp chính** là lớp xuất ra $g^{(t)}$. Nó có vai trò thông thường là phân tích các đầu vào hiện tại $x^{(t)}$ và trạng thái (ngắn hạn) trước đó $h^{(t-1)}$. Trong một ô cơ bản, không có gì khác ngoài lớp này, và đầu ra của nó đi thẳng đến $y^{(t)}$ và $h^{(t)}$. Nhưng trong một ô LSTM, đầu ra của lớp này không đi thẳng ra ngoài; thay vào đó, các phần quan trọng nhất của nó được lưu trữ trong trạng thái dài hạn (và phần còn lại bị loại bỏ).
- **Ba lớp khác** là các **bộ điều khiển cổng (gate controllers)**. Vì chúng sử dụng hàm kích hoạt logistic, các đầu ra nằm trong khoảng từ 0 đến 1. Như bạn có thể thấy, các đầu ra của bộ điều khiển cổng được đưa vào các phép nhân từng phần tử: nếu chúng xuất ra 0, chúng đóng cổng, và nếu chúng xuất ra 1, chúng mở cổng. Cụ thể:
 - **Cổng quên** (được điều khiển bởi $f^{(t)}$) kiểm soát phần nào của trạng thái dài hạn nên bị xóa.
 - **Cổng đầu vào** (được điều khiển bởi $i^{(t)}$) kiểm soát phần nào của $g^{(t)}$ nên được thêm vào trạng thái dài hạn.
 - Cuối cùng, **cổng đầu ra** (được điều khiển bởi $o^{(t)}$) kiểm soát phần nào của trạng thái dài hạn nên được đọc và xuất ra ở bước thời gian này, cả đến $h^{(t)}$ và đến $y^{(t)}$.

Tóm lại, một ô LSTM có thể học cách nhận dạng một đầu vào quan trọng (đó là vai trò của cổng đầu vào), lưu trữ nó trong trạng thái dài hạn, bảo tồn nó miễn là nó cần thiết (đó là vai trò của cổng quên), và trích xuất nó bất cứ khi nào cần. Điều này giải thích tại sao các ô này đã thành công đáng kinh ngạc trong việc nắm bắt các mẫu dài hạn trong chuỗi thời gian, văn bản dài, bản ghi âm và hơn thế nữa.

Phương trình 15-4 tóm tắt cách tính trạng thái dài hạn của ô, trạng thái ngắn hạn của nó và đầu ra của nó ở mỗi bước thời gian cho một thể hiện duy nhất (các phương trình cho toàn bộ một mini-batch rất giống nhau).

Phương trình 15-4. Các phép tính của LSTM

Công thức 15-4: Tính toán LSTM

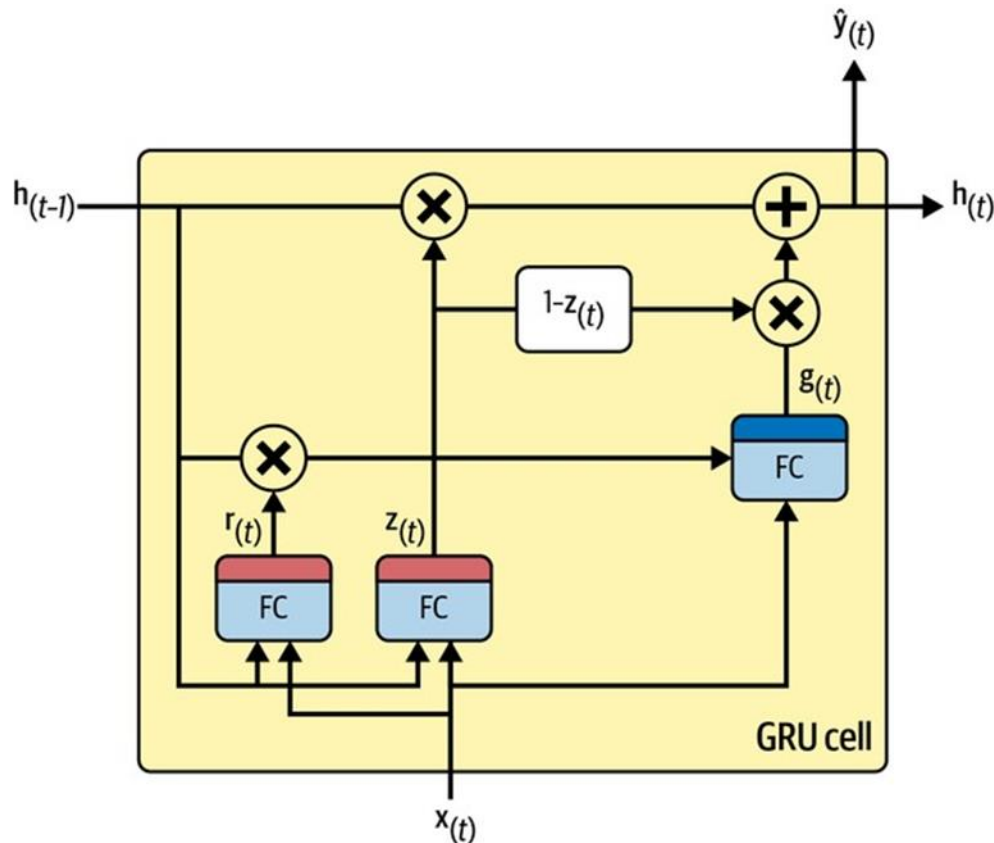
- $i_{(t)} = \sigma(W_{xi}x_{(t)} + W_{hi}h_{(t-1)} + b_i)$
- $f_{(t)} = \sigma(W_{xf}x_{(t)} + W_{hf}h_{(t-1)} + b_f)$
- $o_{(t)} = \sigma(W_{xo}x_{(t)} + W_{ho}h_{(t-1)} + b_o)$
- $g_{(t)} = \tanh(W_{xg}x_{(t)} + W_{hg}h_{(t-1)} + b_g)$
- $c_{(t)} = f_{(t)} \otimes c_{(t-1)} + i_{(t)} \otimes g_{(t)}$
- $y_{(t)} = h_{(t)} = o_{(t)} \otimes \tanh(c_{(t)})$

Trong các phương trình này:

- W_{xi} , W_{xf} , W_{xo} và W_{xg} là các ma trận trọng số của bốn lớp kết nối đến vector đầu vào $x_{(t)}$.
- W_{hi} , W_{hf} , W_{ho} và W_{hg} là các ma trận trọng số của bốn lớp kết nối đến trạng thái ngắn hạn trước đó $h_{(t-1)}$.
- b_i , b_f , b_o và b_g là các số hạng độ lệch cho mỗi trong bốn lớp. TensorFlow khởi tạo b_f là một vector toàn giá trị 1 thay vì 0, điều này giúp ngăn việc quên đi tất cả mọi thứ ngay từ đầu quá trình huấn luyện.

Có một vài biến thể của tế bào LSTM. Một biến thể đặc biệt phổ biến là tế bào GRU, chúng ta sẽ xem xét nó ngay bây giờ.

Các ô GRU Ô đơn vị hồi quy cổng (GRU) (xem Hình 15-13) được Kyunghyun Cho et al. đề xuất trong một bài báo năm 2014 cũng giới thiệu mạng mã hóa-giải mã mà chúng ta đã thảo luận trước đó.



Hình 15-13. Ô GRU

Ô GRU là một phiên bản đơn giản hóa của ô LSTM, và nó dường như hoạt động tốt như nhau (điều này giải thích sự phổ biến ngày càng tăng của nó). Đây là những điểm đơn giản hóa chính:

- Cả hai vector trạng thái được hợp nhất thành một vector duy nhất $h^{(t)}$.
- Một bộ điều khiển cổng duy nhất $z^{(t)}$ kiểm soát cả cổng quên và cổng đầu vào. Nếu bộ điều khiển cổng xuất ra 1, cổng quên mở ($= 1$) và cổng đầu vào đóng ($1 - 1 = 0$). Nếu nó xuất ra 0, điều ngược lại xảy ra. Nói cách khác, bất cứ khi nào một bộ nhớ phải được lưu trữ, vị trí mà nó sẽ được lưu trữ sẽ bị xóa trước. Đây thực sự là một biến thể thường xuyên của ô LSTM.
- Không có cổng đầu ra; toàn bộ vector trạng thái được xuất ra ở mỗi bước thời gian. Tuy nhiên, có một bộ điều khiển cổng mới $r^{(t)}$ kiểm soát phần nào của trạng thái trước đó sẽ được hiển thị cho lớp chính ($g^{(t)}$).

Phương trình 15-5 tóm tắt cách tính trạng thái của ô ở mỗi bước thời gian cho một thể hiện duy nhất.

Công thức 15-5: Tính toán GRU

- $z_{(t)} = \sigma(W_{xz}x_{(t)} + W_{hz}h_{(t-1)} + b_z)$

- $r_{(t)} = \sigma(W_{xr}x_{(t)} + W_{hr}h_{(t-1)} + b_r)$
- $g_{(t)} = \tanh(W_{xg}x_{(t)} + W_{hg}(r_{(t)} \otimes h_{(t-1)}) + b_g)$
- $h_{(t)} = z_{(t)} \otimes h_{(t-1)} + (1 - z_{(t)}) \otimes g_{(t)}$

Keras cung cấp một lớp `tf.keras.layers.GRU`; việc sử dụng nó chỉ đơn giản là thay thế SimpleRNN hoặc LSTM bằng GRU. Nó cũng cung cấp một `tf.keras.layers.GRUCell` trong trường hợp bạn muốn tạo một tế bào tùy chỉnh dựa trên tế bào GRU.

Các tế bào LSTM và GRU là một trong những lý do chính đằng sau sự thành công của các mạng nơ-ron hồi quy (RNN). Tuy nhiên, mặc dù chúng có thể xử lý các chuỗi dài hơn nhiều so với các RNN đơn giản, chúng vẫn có bộ nhớ ngắn hạn khá hạn chế và gặp khó khăn trong việc học các mẫu dài trong chuỗi hơn 100 bước thời gian, chẳng hạn như mẫu âm thanh, chuỗi thời gian dài hoặc câu dài. Một cách để giải quyết vấn đề này là rút ngắn chuỗi đầu vào; ví dụ: sử dụng các lớp tích chập 1D.

Sử dụng các lớp tích chập 1D để xử lý chuỗi

Trong Chương 14, chúng ta đã thấy rằng một lớp tích chập 2D hoạt động bằng cách trượt một số hạt nhân (hoặc bộ lọc) khá nhỏ qua một hình ảnh, tạo ra nhiều bản đồ đặc trưng 2D (một cho mỗi hạt nhân). Tương tự, một **lớp tích chập 1D** trượt một số hạt nhân qua một chuỗi, tạo ra một bản đồ đặc trưng 1D cho mỗi hạt nhân. Mỗi hạt nhân sẽ học cách phát hiện một mẫu tuần tự rất ngắn (không dài hơn kích thước hạt nhân). Nếu bạn sử dụng 10 hạt nhân, thì đầu ra của lớp sẽ bao gồm 10 chuỗi 1D (tất cả có cùng độ dài), hoặc tương đương bạn có thể xem đầu ra này là một chuỗi 10D duy nhất. Điều này có nghĩa là bạn có thể xây dựng một mạng thần kinh bao gồm một sự pha trộn giữa các lớp hồi quy và các lớp tích chập 1D (hoặc thậm chí các lớp gộp 1D). Nếu bạn sử dụng một lớp tích chập 1D với bước nhảy là 1 và padding “same”, thì chuỗi đầu ra sẽ có cùng độ dài với chuỗi đầu vào. Nhưng nếu bạn sử dụng padding “valid” hoặc bước nhảy lớn hơn 1, thì chuỗi đầu ra sẽ ngắn hơn chuỗi đầu vào, vì vậy hãy đảm bảo bạn điều chỉnh các mục tiêu cho phù hợp.

Ví dụ, mô hình sau đây giống như trước đó, ngoại trừ nó bắt đầu bằng một lớp tích chập 1D làm giảm mẫu chuỗi đầu vào theo hệ số 2, sử dụng bước nhảy là 2. Kích thước hạt nhân lớn hơn bước nhảy, vì vậy tất cả các đầu vào sẽ được sử dụng để tính toán đầu ra của lớp, và do đó mô hình có thể học cách bảo toàn thông tin hữu ích, chỉ loại bỏ các chi tiết không quan trọng. Bằng cách rút ngắn các chuỗi, lớp tích chập có thể giúp các lớp GRU phát hiện các mẫu dài hơn, vì vậy chúng ta có thể tăng gấp đôi độ dài chuỗi đầu vào lên 112 ngày. Lưu ý rằng chúng ta cũng phải cắt bỏ ba bước thời gian đầu tiên trong các mục tiêu: thực tế, kích thước hạt nhân là 4, vì vậy đầu ra đầu tiên của lớp tích chập sẽ dựa trên các bước thời gian đầu vào 0 đến 3, và các dự báo đầu tiên sẽ cho các bước thời gian 4 đến 17 (thay vì các bước thời gian 1 đến 14). Hơn nữa, chúng ta phải giảm mẫu các mục tiêu theo hệ số 2, do bước nhảy:

```
conv_rnn_model = tf.keras.Sequential([
    tf.keras.layers.Conv1D(filters=32, kernel_size=4, strides=2,
                           activation="relu", input_shape=[None, 5]),
    tf.keras.layers.GRU(32, return_sequences=True),
```

```

    tf.keras.layers.Dense(14)
])

# Các hàm và dữ liệu cần thiết đã được định nghĩa ở trên (mulvar_train, to_seq2seq_dataset)
longer_train = to_seq2seq_dataset(mulvar_train, seq_length=112,
                                  shuffle=True, seed=42)
longer_valid = to_seq2seq_dataset(mulvar_valid, seq_length=112)

# Điều chỉnh mục tiêu do Conv1D
# Các giá trị mục tiêu Y đã được tạo với ahead=14.
# Y[:, 3::2] là cần thiết vì:
# 1. kernel_size=4, strides=2 nghĩa là đầu ra đầu tiên của Conv1D tương ứng với input [0,1,2,3].
#    Vậy, dự đoán cho Y_t+1, Y_t+2, ..., Y_t+14 sẽ cần bắt đầu từ input_time_step 0.
# 2. Sau Conv1D, mỗi output của Conv1D tương ứng với 2 time steps trong input gốc.
#    Output của Dense layer sau GRU sẽ dự đoán 14 bước.
#    Y_seq2seq_ds có dạng (X_window, Y_window_sequence).
#    Y_window_sequence có shape (batch_size, seq_length, ahead).
#    Khi seq_length=112, và ahead=14, Y_window_sequence có shape (batch_size, 112, 14).
#    Stride=2 nghĩa là ta chỉ cần 112/2 = 56 outputs từ GRU layer.
#    Mỗi output của GRU layer dự đoán 14 bước.
#    Điều chỉnh Y[:, 3::2] là để căn chỉnh mục tiêu với các dự đoán sau khi Conv1D giảm mẫu.
downsampled_train = longer_train.map(lambda X, Y: (X, Y[:, 3::2]))
downsampled_valid = longer_valid.map(lambda X, Y: (X, Y[:, 3::2]))

# Biên dịch và huấn luyện mô hình
conv_rnn_model.compile(loss=tf.keras.losses.Huber(), optimizer=opt, metrics=["mae"])
history_conv_rnn = conv_rnn_model.fit(downsampled_train, validation_data=downsampled_valid, epochs=500, callbacks=[early_stopping_cb])

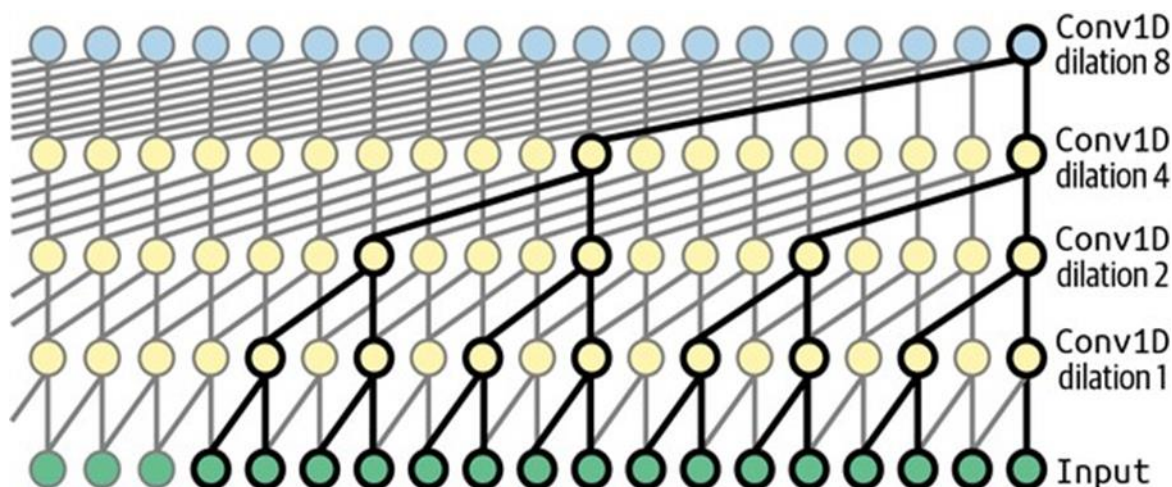
```

Nếu bạn huấn luyện và đánh giá mô hình này, bạn sẽ thấy rằng nó vượt trội hơn mô hình trước đó (với một biên độ nhỏ). Trên thực tế, có thể chỉ sử dụng các lớp tích chập 1D và loại bỏ hoàn toàn các lớp hồi quy!

WaveNet

Trong một bài báo năm 2016, Aaron van den Oord và các nhà nghiên cứu DeepMind khác đã giới thiệu một kiến trúc mới lạ có tên **WaveNet**. Họ chồng các lớp tích chập 1D lên nhau, tăng gấp đôi tỷ lệ giãn nở (khoảng cách giữa các đầu vào của mỗi nơ-ron) ở mỗi lớp: lớp tích chập đầu tiên chỉ nhìn thấy hai bước thời gian tại một thời điểm, trong khi lớp tiếp theo nhìn thấy bốn bước thời gian (trường thụ cảm của nó dài bốn bước thời gian), lớp tiếp theo nhìn thấy tám bước thời gian, v.v. (xem Hình 15-14). Bằng cách này, các lớp thấp hơn học các

mẫu ngắn hạn, trong khi các lớp cao hơn học các mẫu dài hạn. Nhờ tỷ lệ giãn nở tăng gấp đôi, mạng có thể xử lý các chuỗi cực kỳ lớn rất hiệu quả.



Hình 15-14. Kiến trúc WaveNet

Các tác giả của bài báo thực sự đã chèn 10 lớp tích chập với tỷ lệ giãn nở là 1, 2, 4, 8, ..., 256, 512, sau đó họ chèn một nhóm 10 lớp giống hệt khác (cũng với tỷ lệ giãn nở 1, 2, 4, 8, ..., 256, 512), sau đó lại một nhóm 10 lớp giống hệt khác. Họ giải thích kiến trúc này bằng cách chỉ ra rằng một chèn 10 lớp tích chập với các tỷ lệ giãn nở này sẽ hoạt động như một lớp tích chập siêu hiệu quả với kích thước hạt nhân 1.024 (ngoại trừ nhanh hơn nhiều, mạnh hơn và sử dụng ít tham số hơn đáng kể). Họ cũng đệm đầu vào các chuỗi bằng một số lượng số không bằng với tỷ lệ giãn nở trước mỗi lớp, để giữ cùng độ dài chuỗi trong toàn bộ mạng.

Đây là cách triển khai một WaveNet đơn giản hóa để giải quyết cùng các chuỗi như trước đây: Với WaveNet, chúng ta sẽ cần xây dựng một mô hình phức tạp hơn một chút bằng cách sử dụng API chức năng của Keras (hoặc kế thừa từ `tf.keras.Model`). Cấu trúc cơ bản sẽ bao gồm các lớp Conv1D xếp chồng lên nhau với tỷ lệ giãn nở tăng dần. Để giữ cùng độ dài chuỗi, chúng ta sẽ sử dụng `padding="causal"` (đảm bảo đầu ra tại thời điểm t chỉ phụ thuộc vào đầu vào tại thời điểm t và các thời điểm trước đó, phù hợp với chuỗi thời gian).

Dưới đây là cách triển khai một WaveNet đơn giản hóa trong Keras để giải quyết nhiệm vụ dự báo chuỗi:

```
import tensorflow as tf
import pandas as pd
import numpy as np

# Giả định df, df_mulvar, mulvar_train, mulvar_valid, to_seq2seq_dataset,
# seq_length, ahead, opt, early_stopping_cb đã được định nghĩa ở các bước trư
# ớc.

# Chuẩn bị dữ liệu cho WaveNet
# WaveNet thường yêu cầu một receptive field lớn, nên seq_length cần đủ lớn.
# Ví dụ, nếu chúng ta dùng 10 lớp với dilation_rate từ 1 đến 512, receptive f
```

```

ield là 1023
# (2^10 - 1). Chúng ta sẽ dùng seq_length = 2 * (2**num_dilation_layers - 1)
để đảm bảo
# đủ dữ liệu cho kernel_size = 2.
num_dilation_layers = 10 # Số lớp giãn nở
wavenet_seq_length = 2 * (2**num_dilation_layers - 1) + 1 # +1 cho target đầu
tiên, đảm bảo padding
wavenet_ahead = ahead # Vẫn dự báo 14 bước
wavenet_target_col = 1 # Cột 'rail'

wavenet_train_ds = to_seq2seq_dataset(mulvar_train.to_numpy(), seq_length=wav
enet_seq_length,
                                ahead=wavenet_ahead, target_col=wavenet
_target_col,
                                shuffle=True, seed=42)
wavenet_valid_ds = to_seq2seq_dataset(mulvar_valid.to_numpy(), seq_length=wav
enet_seq_length,
                                ahead=wavenet_ahead, target_col=wavenet
_target_col)

# Xây dựng mô hình WaveNet
inputs = tf.keras.layers.Input(shape=[None, 5]) # Input shape [batch, time_st
eps, features]
x = inputs

# Tương tự như trong bài báo gốc WaveNet, sử dụng gated activations (sigmoid
* tanh)
# Tuy nhiên, để đơn giản, chúng ta sẽ sử dụng ReLU và Dense cho các kết nối c
òn lại.
# Mô hình dưới đây là một phiên bản đơn giản hóa, tập trung vào Dilated Convo
lutions.

# Lớp Conv1D đầu tiên
x = tf.keras.layers.Conv1D(filters=32, kernel_size=2, padding="causal", activ
ation="relu")(x)

# Các lớp tích chập giãn nở
skip_connections = []
for i in range(num_dilation_layers):
    dilation_rate = 2**i
    conv_layer = tf.keras.layers.Conv1D(filters=32, kernel_size=2,
                                padding="causal",
                                dilation_rate=dilation_rate,
                                activation="relu")

    x = conv_layer(x)
    skip_connections.append(x) # Thu thập skip connections

# Kết hợp các skip connections và thêm Dense layer
x = tf.keras.layers.Add()(skip_connections) # Tổng hợp các đầu ra từ các lớp

```

```

giãn nở
x = tf.keras.layers.Conv1D(filters=32, kernel_size=1, activation="relu")(x) #
Conv1D 1x1
outputs = tf.keras.layers.Dense(wavenet_ahead)(x[:, -1, :]) # Chỉ lấy đầu ra
cuối cùng cho dự báo

wavenet_model = tf.keras.Model(inputs=inputs, outputs=outputs)

# Biên dịch và huấn luyện mô hình
wavenet_model.compile(loss=tf.keras.losses.Huber(), optimizer=opt, metrics=["
mae"])

# Chú ý: Việc huấn luyện WaveNet có thể tốn thời gian và tài nguyên hơn.
# Bạn có thể điều chỉnh số lượng epoch và patience của early_stopping_cb.
history_wavenet = wavenet_model.fit(wavenet_train_ds, validation_data=wavenet
_valid_ds, epochs=500, callbacks=[early_stopping_cb])

```

Trong triển khai này:

- **tf.keras.layers.Input**: Định nghĩa đầu vào của mô hình.
- **Conv1D với padding="causal"**: Đảm bảo rằng mỗi đầu ra tại bước thời gian t chỉ phụ thuộc vào các đầu vào tại hoặc trước t . Điều này rất quan trọng đối với các chuỗi thời gian để tránh “nhìn thấy” dữ liệu trong tương lai.
- **dilation_rate**: Tăng gấp đôi ở mỗi lớp ($2 \times i$) để mở rộng trường thụ cảm một cách hiệu quả, cho phép mô hình nắm bắt các phụ thuộc dài hạn với ít lớp hơn.
- **skip_connections**: Các đầu ra từ mỗi lớp giãn nở được thu thập và cộng lại (sử dụng `tf.keras.layers.Add()`). Điều này giúp cải thiện luồng gradient và cho phép mô hình truy cập trực tiếp các đặc trưng ở các cấp độ trừu tượng khác nhau.
- **Conv1D với kernel_size=1**: Được sử dụng sau các kết nối bỏ qua để tổng hợp các thông tin trước khi đưa vào lớp đầu ra cuối cùng.
- **Dense(wavenet_ahead)**: Lớp Dense cuối cùng để tạo ra dự báo cho wavenet_ahead (14) bước thời gian tiếp theo. `x[:, -1, :]` được sử dụng để chỉ lấy đầu ra của bước thời gian cuối cùng từ chuỗi đầu ra của Conv1D trước đó, biến nó thành mô hình chuỗi-sang-vector dự báo nhiều bước.

Mô hình này là một phiên bản đơn giản hóa của WaveNet gốc (ví dụ, nó không sử dụng “gated activations” sigmoid-tanh phức tạp hoặc các khối dư), nhưng nó thể hiện ý tưởng cốt lõi của việc sử dụng dilated convolutions để xử lý các chuỗi dài.

```

wavenet_model = tf.keras.Sequential()
wavenet_model.add(tf.keras.layers.Input(shape=[None, 5]))

for rate in (1, 2, 4, 8) * 2:
    wavenet_model.add(tf.keras.layers.Conv1D( filters=32,
kernel_size=2, padding="causal", activation="relu", dilation_rate=rate))
wavenet_model.add(tf.keras.layers.Conv1D(filters=14, kernel_size=1))

```


Mô hình Sequential này bắt đầu với một lớp đầu vào được chỉ định rõ ràng—điều này đơn giản hơn là cố gắng chỉ đặt `input_shape` trên lớp đầu tiên. Sau đó, nó tiếp tục với một lớp tích chập 1D sử dụng padding “causal”, giống như padding “same” ngoại trừ việc các số 0 chỉ được thêm vào đầu chuỗi đầu vào, thay vì ở cả hai bên. Điều này đảm bảo rằng lớp tích chập không “nhìn trộm” vào tương lai khi đưa ra dự đoán. Sau đó, chúng ta thêm các cặp lớp tương tự sử dụng tỷ lệ giãn nở tăng dần: 1, 2, 4, 8, và lặp lại 1, 2, 4, 8. Cuối cùng, chúng ta thêm lớp đầu ra: một lớp tích chập với 14 bộ lọc kích thước 1 và không có bất kỳ hàm kích hoạt nào. Như chúng ta đã thấy trước đó, một lớp tích chập như vậy tương đương với một lớp Dense với 14 đơn vị. Nhờ có padding causal, mỗi lớp tích chập xuất ra một chuỗi có cùng độ dài với chuỗi đầu vào của nó, vì vậy các mục tiêu chúng ta sử dụng trong quá trình huấn luyện có thể là toàn bộ các chuỗi dài 112 ngày: không cần cắt hoặc giảm mẫu chúng.

Các mô hình chúng ta đã thảo luận trong phần này mang lại hiệu suất tương tự cho nhiệm vụ dự báo lượng hành khách, nhưng chúng có thể thay đổi đáng kể tùy thuộc vào nhiệm vụ và lượng dữ liệu có sẵn. Trong bài báo WaveNet, các tác giả đã đạt được hiệu suất tiên tiến trên nhiều nhiệm vụ âm thanh khác nhau (do đó có tên kiến trúc), bao gồm các nhiệm vụ chuyển văn bản thành giọng nói, tạo ra giọng nói cực kỳ chân thực trên nhiều ngôn ngữ. Họ cũng sử dụng mô hình để tạo nhạc, từng mẫu âm thanh một. Thành tựu này càng ấn tượng hơn khi bạn nhận ra rằng một giây âm thanh có thể chứa hàng chục nghìn bước thời gian—ngay cả LSTM và GRU cũng không thể xử lý các chuỗi dài như vậy.

Với những điều đó, bây giờ bạn có thể giải quyết tất cả các loại chuỗi thời gian! Trong Chương 16, chúng ta sẽ tiếp tục khám phá RNN, và chúng ta sẽ xem chúng có thể giải quyết nhiều nhiệm vụ NLP khác nhau như thế nào.

Bài tập

1. Bạn có thể nghĩ ra một vài ứng dụng cho RNN chuỗi-sang-chuỗi không? Thế còn RNN chuỗi-sang-vector, và RNN vector-sang-chuỗi thì sao?
2. Đầu vào của một lớp RNN phải có bao nhiêu chiều? Mỗi chiều đại diện cho điều gì? Còn đầu ra của nó thì sao?
3. Nếu bạn muốn xây dựng một RNN sâu chuỗi-sang-chuỗi, những lớp RNN nào nên có `return_sequences=True`? Thế còn RNN chuỗi-sang-vector thì sao?
4. Giả sử bạn có một chuỗi thời gian đơn biến hàng ngày, và bạn muốn dự báo bảy ngày tiếp theo. Bạn nên sử dụng kiến trúc RNN nào?
5. Những khó khăn chính khi huấn luyện RNN là gì? Bạn có thể xử lý chúng như thế nào?
6. Bạn có thể phá vỡ kiến trúc của ô LSTM không?
7. Tại sao bạn lại muốn sử dụng các lớp tích chập 1D trong một RNN?
8. Bạn có thể sử dụng kiến trúc mạng thần kinh nào để phân loại video?
9. Huấn luyện một mô hình phân loại cho tập dữ liệu SketchRNN, có sẵn trong TensorFlow Datasets.
10. Tải tập dữ liệu Bach chorales và giải nén nó. Nó bao gồm 382 hợp xướng do Johann Sebastian Bach sáng tác. Mỗi hợp xướng dài từ 100 đến 640 bước thời gian, và mỗi bước thời gian chứa 4 số nguyên, trong đó mỗi số nguyên tương ứng với chỉ số của

một nốt nhạc trên đàn piano (ngoại trừ giá trị 0, có nghĩa là không có nốt nhạc nào được chơi). Huấn luyện một mô hình—hồi quy, tích chập hoặc cả hai—có thể dự đoán bước thời gian tiếp theo (bốn nốt nhạc), dựa trên một chuỗi các bước thời gian từ một hợp xướng. Sau đó sử dụng mô hình này để tạo nhạc giống Bach, từng nốt một: bạn có thể làm điều này bằng cách cung cấp cho mô hình phần đầu của một hợp xướng và yêu cầu nó dự đoán bước thời gian tiếp theo, sau đó nối các bước thời gian này vào chuỗi đầu vào và yêu cầu mô hình nốt tiếp theo, v.v. Cũng hãy nhớ xem mô hình Coconet của Google, đã được sử dụng cho một hình vẽ nguệch ngoạc Google đẹp mắt về Bach.

Các giải pháp cho các bài tập này có sẵn ở cuối sổ tay chương này, tại <https://homl.info/colab3>.

1 Lưu ý rằng nhiều nhà nghiên cứu thích sử dụng hàm kích hoạt hyperbolic tangent (tanh) trong RNN hơn là hàm kích hoạt ReLU. Ví dụ, xem bài báo năm 2013 của Vu Pham et al. “Dropout Improves Recurrent Neural Networks for Handwriting Recognition”. RNN dựa trên ReLU cũng có thể thực hiện được, như được thể hiện trong bài báo năm 2015 của Quoc V. Le et al. “A Simple Way to Initialize Recurrent Networks of Rectified Linear Units”. 2 Nal Kalchbrenner và Phil Blunsom, “Recurrent Continuous Translation Models”, Kỷ yếu Hội nghị về Phương pháp thực nghiệm trong Xử lý ngôn ngữ tự nhiên năm 2013 (2013): 1700–1709. 3 Dữ liệu mới nhất từ Cơ quan Giao thông Chicago có sẵn tại Cổng dữ liệu Chicago. 4 Có những cách tiếp cận nguyên tắc hơn để chọn các siêu tham số tốt, dựa trên phân tích hàm tự tương quan (ACF) và hàm tự tương quan riêng phần (PACF), hoặc giảm thiểu các chỉ số AIC hoặc BIC (được giới thiệu trong Chương 9) để phạt các mô hình sử dụng quá nhiều tham số và giảm nguy cơ quá khớp dữ liệu, nhưng tìm kiếm lưới là một nơi tốt để bắt đầu. Để biết thêm chi tiết về cách tiếp cận ACF-PACF, hãy xem bài đăng rất hay này của Jason Brownlee. 5 Lưu ý rằng giai đoạn xác thực bắt đầu vào ngày 1 tháng 1 năm 2019, vì vậy dự đoán đầu tiên là cho ngày 26 tháng 2 năm 2019, tám tuần sau đó. Khi chúng ta đánh giá các mô hình đường cơ sở, chúng ta đã sử dụng các dự đoán bắt đầu vào ngày 1 tháng 3, nhưng điều này đủ gần. 6 Cứ thoải mái thử nghiệm với mô hình này. Ví dụ, bạn có thể thử dự báo cả lượng hành khách xe buýt và đường sắt trong 14 ngày tiếp theo. Bạn sẽ cần điều chỉnh các mục tiêu để bao gồm cả hai, và làm cho mô hình của bạn xuất ra 28 dự báo thay vì 14. 7 César Laurent et al., “Batch Normalized Recurrent Neural Networks”, Kỷ yếu Hội nghị quốc tế IEEE về Âm học, Lời nói và Xử lý tín hiệu (2016): 2657–2661. 8 Jimmy Lei Ba et al., “Layer Normalization”, arXiv preprint arXiv:1607.06450 (2016).

9 Sẽ đơn giản hơn nếu kế thừa từ SimpleRNNCell thay vì vậy để chúng ta không phải tạo một SimpleRNNCell nội bộ hoặc xử lý các thuộc tính state_size và output_size, nhưng mục tiêu ở đây là để cho thấy cách tạo một ô tùy chỉnh từ đầu. 10 Một nhân vật trong các bộ phim hoạt hình *Tìm kiếm Nemo* và *Tìm kiếm Dory* bị mất trí nhớ ngắn hạn. 11 Sepp Hochreiter và Jürgen Schmidhuber, “Long Short-Term Memory”, *Neural Computation* 9, số 8 (1997): 1735–1780. 12 Haşim Sak et al., “Long Short-Term Memory Based Recurrent Neural Network Architectures for Large Vocabulary Speech Recognition”, arXiv preprint arXiv:1402.1128 (2014). 13 Wojciech Zaremba et al., “Recurrent Neural Network Regularization”, arXiv preprint arXiv:1409.2329 (2014). 14 Kyunghyun Cho et al., “Learning Phrase

Representations Using RNN Encoder–Decoder for Statistical Machine Translation”, Kỷ yếu Hội nghị về Phương pháp thực nghiệm trong Xử lý ngôn ngữ tự nhiên năm 2014 (2014): 1724–1734. 15 Xem Klaus Greff et al., “LSTM: A Search Space Odyssey”, *IEEE Transactions on Neural Networks and Learning Systems* 28, số 10 (2017): 2222–2232. Bài báo này dường như cho thấy tất cả các biến thể LSTM đều có hiệu suất gần như nhau. 16 Aaron van den Oord et al., “WaveNet: A Generative Model for Raw Audio”, arXiv preprint arXiv:1609.03499 (2016). 17 WaveNet hoàn chỉnh sử dụng thêm một vài thủ thuật, chẳng hạn như skip connections giống như trong ResNet, và các đơn vị kích hoạt có cổng tương tự như những gì được tìm thấy trong một ô GRU. Xem sổ tay chương này để biết thêm chi tiết.